The Open Group Preliminary Standard

O-PAS™ Standard, Version 2.1

Part 6.2 – Information and Exchange Models: Basic Configuration

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Copyright © 2021, The Open Group

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without the prior permission of the copyright owners.

Any use of this publication for commercial purposes is subject to the terms of the Annual Commercial License relating to it. For further information, see www.opengroup.org/legal/licensing.

The Open Group Preliminary Standard

O-PAS™ Standard, Version 2.1: Part 6.2 – Information and Exchange Models: Basic Configuration

Document Number: P210-6b

Published by The Open Group, May 2021.

Comments relating to the material contained in this document may be submitted to:

The Open Group, Apex Plaza, Forbury Road, Reading, Berkshire, RG1 1AX, United Kingdom

or by electronic mail to:

ogspecs@opengroup.org

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Contents

1 Introduction 1
- 1.1 Objective 1
- 1.2 Overview 1
- 1.3 Conformance 1
- 1.4 Normative References 1
- 1.5 Terminology 3
- 1.6 Future Directions (Informative) 3

2 Terms and Definitions 4
- 2.1 OPAF Glossary and Abbreviations 4

3 Information Model 5
- 3.1 Information Model 5
- 3.2 O-PAS Signals 7
- 3.3 O-PAS Function Block 10
- 3.4 Basic Model Configuration Constraints 13

4 Import/Export Format 14
- 4.1 Import/Export Format Structure 14
- 4.2 O-PAS Common Objects 14
  - 4.2.1 Basic-Project Role Class 15
  - 4.2.2 UA-Tag Role Class 15
  - 4.2.3 UA-TagVariable Role Class 15
  - 4.2.4 Basic-Signal Role Class 15
  - 4.2.5 Basic-SignalInterface Interface Class 15
  - 4.2.6 Basic-TriggerInterface 16
  - 4.2.7 Basic-TagList Role Class 16
  - 4.2.8 Basic-VendorNameplate Attribute Type 16
  - 4.2.9 Basic-TagNameplate Attribute Type 16
  - 4.2.10 Basic-DCN Role Class 16
  - 4.2.11 Basic-DCP Role Class 16
  - 4.2.12 Basic-ControlApplication Role Class 17
  - 4.2.13 Basic-FunctionBlockApplication Role Class 17
  - 4.2.14 Basic-BaseEngine Role Class 17
  - 4.2.15 Basic-FBExecutionEngine Role Class 17
  - 4.2.16 Basic-SignalService Role Class 18
  - 4.2.17 Basic-IOServiceEngine Role Class 18
  - 4.2.18 Basic-FunctionBlockGroup Role Class 18
  - 4.2.19 Basic-FunctionBlock Role Class 18
  - 4.2.20 Basic-OCFB_IN System Unit Class 19
  - 4.2.21 Basic-OCFB_OUT System Unit Class 19
  - 4.2.22 UA-UOMLibrary Role Class 19
  - 4.2.23 UA-UOMNamespace Role Class 20
  - 4.2.24 UA-UnitOfMeasure Role Class 20
  - 4.2.25 UA-UOMLink Interface Class 20
- 4.3 Import/Export Profile 20

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

4.3.1 OBC-201: Basic Configuration Import/Export Profile 20
5 Download/Upload 21
6 OCF Information Model 22
6.1 O-PAS Information Model 22
6.2 OPC UA Information Model PA-DIM Overview 24
6.3 O-PAS OPC UA Function Block Information Model 25
6.4 O-PAS OPC UA ObjectTypes 26
6.4.1 IDCNBaseType 26
6.4.2 DCNBaseType 27
6.4.3 OPASBaseEngineType 28
6.4.4 ISignalServiceType 29
6.4.5 IOServiceEngineType 29
6.4.6 FBGroupType 30
6.4.7 FBExecutionEngineType 30
6.4.8 FunctionBlockType 31
6.4.9 FBErrorType 35
6.4.10 FBOptionsType 35
6.4.11 OCFB_Type 38
6.4.12 OCFB_OutType 38
6.4.13 OCFB_InType 39
6.5 O-PAS OPC UA VariableTypes 39
6.5.1 TriggerType 39
6.5.2 OPASAnalogSignalType 40
6.5.3 OPASAnalogSimulationSignalType 41
6.5.4 AnalogUnitRangeScaleType 41
6.5.5 FBModeType 42
6.6 O-PAS OPC UA ReferenceTypes 43
6.6.1 ConnectedToOutput 43
6.7 Instance AddressSpace 44
6.7.1 OPAS Folder and SignalType Alias 44
6.7.2 FunctionBlocks Aliases 44
6.8 Profiles and Facets 46
6.8.1 OBC-F-201: O-PAS Base Function Block Facet 46
6.8.2 OBC-202: O-PAS I/O Server Profile 46
6.8.3 OBC-203: O-PAS Function Block Compute Server Profile 47
7 Namespaces 50
7.1 Namespace Metadata 50
7.2 Handling of OPC UA Namespaces 50
A Connectivity Framework Namespace and Mappings (Normative) 53
A.1 Namespace and Identifiers for the Connectivity Framework Information Model 53
B O-PAS AutomationML Definitions (Normative) 54
C Future Extension to IOServiceEngineType (Informative) 55
D Abbreviations 56

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Preface

The Open Group

The Open Group is a global consortium that enables the achievement of business objectives through technology standards. Our diverse membership of more than 800 organizations includes customers, systems and solutions suppliers, tools vendors, integrators, academics, and consultants across multiple industries.

The mission of The Open Group is to drive the creation of Boundaryless Information Flow™ achieved by:

- Working with customers to capture, understand, and address current and emerging requirements, establish policies, and share best practices
- Working with suppliers, consortia, and standards bodies to develop consensus and facilitate interoperability, to evolve and integrate specifications and open source technologies
- Offering a comprehensive set of services to enhance the operational efficiency of consortia
- Developing and operating the industry's premier certification service and encouraging procurement of certified products

Further information on The Open Group is available at www.opengroup.org.

The Open Group publishes a wide range of technical documentation, most of which is focused on development of Standards and Guides, but which also includes white papers, technical studies, certification and testing documentation, and business titles. Full details and a catalog are available at www.opengroup.org/library.

This Document

This document is Part 6.2 of the O-PAS™ Standard, Version 2.1, a Preliminary Standard of The Open Group. It has been developed and approved by The Open Group.

The O-PAS Standard consists of the following 12 parts (of the anticipated 13 parts):

- O-PAS Part 1 – Technical Architecture Overview (Informative)
- O-PAS Part 2 – Security
- O-PAS Part 3 – Profiles (Informative)
- O-PAS Part 4 – O-PAS Connectivity Framework (OCF)
- O-PAS Part 5 – System Management
- O-PAS Part 6.1 – Information and Exchange Models: Overview and Interfaces

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

- O-PAS Part 6.2 – Information and Exchange Models: Basic Configuration (this document)
- O-PAS Part 6.3 – Information and Exchange Models: Alarm and Events Configuration
- O-PAS Part 6.4 – Information and Exchange Models: Function Blocks
- O-PAS Part 6.5 – Information and Exchange Models: IEC 61499
- O-PAS Part 6.6 – Information and Exchange Models: IEC 61131
- O-PAS Part 7 – Distributed Control Platform

The parts listed above are separate documents that can be updated and re-versioned as required as we move forward with the O-PAS Standard.

The O-PAS Standard, Version 2.1 is being published initially as a Preliminary Standard since it addresses an emerging area of technology; therefore, it may change before being published as a full standard of The Open Group. In such a case it will be made as upwards-compatible as possible with the corresponding Preliminary Standard, but complete upwards-compatibility is not guaranteed.

Once this document becomes a full standard, it will supersede the O-PAS Standard, Version 1.0 published in December 2019.

Conventions

A Glossary and Abbreviations reference is available. If a term is not defined in that document then the common English definition, as defined in Merriam-Webster's Collegiate Dictionary, applies.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Trademarks

ArchiMate, DirecNet, Making Standards Work, Open O logo, Open O and Check Certification logo, Platform 3.0, The Open Group, TOGAF, UNIX, UNIXWARE, and the Open Brand X logo are registered trademarks and Boundaryless Information Flow, Build with Integrity Buy with Confidence, Commercial Aviation Reference Architecture, Dependability Through Assuredness, Digital Practitioner Body of Knowledge, DPBoK, EMMM, FACE, the FACE logo, FHIM Profile Builder, the FHIM logo, FPB, Future Airborne Capability Environment, IT4IT, the IT4IT logo, O-AA, O-DEF, O-HERA, O-PAS, Open Agile Architecture, Open FAIR, Open Footprint, Open Process Automation, Open Subsurface Data Universe, Open Trusted Technology Provider, OSDU, Sensor Integration Simplified, SOSA, and the SOSA logo are trademarks of The Open Group.

PA-DIM is a trademark of FieldComm Group, Inc.

UML is a registered trademark of Object Management Group, Inc. in the United States and/or other countries.

All other brands, company, and product names are used for identification purposes only and may be trademarks that are the sole property of their respective owners.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Acknowledgements

The Open Group gratefully acknowledges the International Society of Automation for use of the ISA copyrighted ANSI/ISA 62443 series of standards. Visit www.isa.org.

The Open Group gratefully acknowledges and thanks the OPC Foundation for the template used to develop an OPC UA Companion Specification that is included in this O-PAS Standard.

The Open Group gratefully acknowledges the contribution of the following people in the development of this document:

- David Fort – Rockwell Automation
- Kirk Hansen – Enterprise Transformation Partners
- Paul Hunkar – Yokogawa Electric Corporation
- Alex Johnson – Schneider Electric
- Vien Nguyen – Yokogawa Electric Corporation
- Lars Ott – CODESYS Group
- Sølve Raaen – Kongsberg Maritime
- Jeremy Rover – Intel Corporation

The Open Group gratefully acknowledges the contribution and leadership of the Co-Chairs of the Open Process Automation™ Forum and Working Groups.

Forum Co-Chairs

- Don Bartusiak – Collaborative Systems Integration, LLC
- Trevor Cusworth – Schneider Electric

Business Working Group Co-Chair

- Luis Duran – ABB Automation GmbH
- Jacco Opmeer – Shell

Enterprise Architecture Work Grouping Co-Chairs

- Dave Emerson – Yokogawa Electric Corporation
- Harirajan Padmanabhan – hpad, Inc.

Technology Working Group Co-Chairs

- David Fort – Rockwell Automation

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

- Kirk Smith – Intel Corporation

Certification Working Group Co-Chairs

- Ed Agis – Intel Corporation
- Ron Breault – Wind River Systems, Inc.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Referenced Documents

The following documents are referenced in Part 6.2 of the O-PAS Standard.

The documents are referred to in the text in such a way that some or all of their content constitutes requirements of this part. For dated references, only the version cited applies. For undated references, the latest version of the referenced document (including any amendments) applies.

(Please note that the links below are good at the time of writing but cannot be guaranteed for the future.)

Normative References

Normative references for Part 6.2 of the O-PAS Standard are defined in Section 1.4.

Informative References

- IEC 61987: Industrial-Process Measurement and Control – Data Structures and Elements in Process Equipment Catalogs

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

1 Introduction

1.1 Objective

The objective of Part 6.2 of the O-PAS Standard is to define the basic information and exchange models that enable the portability of configuration information and "Layer-F" applications between O-PAS conformant hardware and software (see O-PAS Part 1 – Technical Architecture Overview (Informative) for definitions of different O-PAS application layers).

1.2 Overview

Part 6.2 defines the basic information and exchange models for project, Distributed Control Node (DCN), automation signals and associated Input/Output (I/O) mappings. It also defines supported profiles. This document includes:

- Chapter 3: a definition of a UML® class model that is used to define the implementation models
- Chapter 4: a definition of an import/export format that is used to contain the basic configuration information managed by an O-PAS conformant product, that can be extended by subsequent parts of the O-PAS Standard
- Chapter 5: a definition of a download/upload model that is used to download and upload the configuration information to an O-PAS conformant product
- Chapter 6: a reference to the information model for the OCF (defined in O-PAS Part 4 – O-PAS Connectivity Framework (OCF)) that is used in the operation of the O-PAS conformant product

1.3 Conformance

Part 6.2 of the O-PAS Standard is normative, and conformance requirements (marked with "CRQ" identifiers) are defined in Chapters 3, 4, 6, and 7.

1.4 Normative References

The following standards contain provisions which, through references in this document, constitute provisions of Part 6.2 of the O-PAS Standard. At the time of publication, the editions indicated were valid. All standards are subject to revision, and parties to agreements based on this document are encouraged to investigate the possibility of applying the most recent editions of the standards listed below.

- ANSI/ISA-62443-4-2:2018: Security for Industrial Automation and Control Systems – Part 4-2: Technical Security Requirements for IACS Components (adopted by IEC as IEC 62443-4-2)

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

- Extensible Markup Language (XML) 1.0 (Third Edition), W3C Recommendation 04 February 2004, available at http://www.w3.org/TR/2004/REC-xml-20040204/
- IEC 61131-3:2013. Programmable Controllers – Part 3: Programming Languages; refer to: https://webstore.iec.ch/publication/4552
- IEC 61131-10:2019: Programmable Controllers – Part 10: PLC Open XML Exchange Format; refer to: https://webstore.iec.ch/publication/33034
- IEC 61499-1:2012: Function Blocks – Part 1: Architecture; refer to: https://webstore.iec.ch/publication/5506
- IEC 62714-1:2018: Engineering Data Exchange Format for Use in Industrial Automation Systems Engineering – Automation Markup Language – Part 1: Architecture and General Requirements; refer to: https://webstore.iec.ch/publication/32339
- IETF RFC 2119: Key Words for Use in RFCs to Indicate Requirement Levels, March 1997; refer to: https://tools.ietf.org/html/rfc2119
- ISO/IEC 29500-2: Information Technology – Document Description and Processing Languages – Office Open XML File Formats – Part 2: Open Packaging Conventions
- O-PAS Standard, Version 2.1: Part 1 – Technical Architecture Overview (Informative), The Open Group Preliminary Standard (P210-1), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 2 – Security, The Open Group Preliminary Standard (P210-2), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 4 – O-PAS Connectivity Framework (OCF), The Open Group Preliminary Standard (P210-4), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 6.1 – Information and Exchange Models: Overview and Interfaces, The Open Group Preliminary Standard (P210-6a), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 6.3 – Information and Exchange Models: Alarm and Events Configuration, The Open Group Preliminary Standard (P210-6c), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 6.4 – Information and Exchange Models: Function Blocks, The Open Group Preliminary Standard (P210-6d), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 6.5 – Information and Exchange Models: IEC 61499, The Open Group Preliminary Standard (P210-6e), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 6.6 – Information and Exchange Models: IEC 61131, The Open Group Preliminary Standard (P210-6f), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

- O-PAS Standard, Version 2.1: Part 7 – Distributed Control Platform, The Open Group Preliminary Standard (P210-7), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- Unified Architecture (UA) Specification, Part 5 (OPC 10000-5): Information Model (also IEC 62541-5), OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part5/
- Unified Architecture (UA) Specification, Part 6 (OPC 10000-6): Mappings, OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part6/
- Unified Architecture (UA) Specification, Part 7 (OPC 10000-7): Profiles, OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part7/
- Unified Architecture (UA) Specification, Part 8 (OPC 10000-8): Data Access (also IEC 62541-8:2015), OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part8/
- Unified Architecture (UA) Specification, Part 17 (OPC 10000-17): Alias Names (also IEC 62541-17), OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part17/
- Unified Architecture (UA) Specification, Part 100 (OPC 10000-100): Devices (also IEC 62541-100), OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part100/
- Unified Architecture (UA) Specification (OPC 30081): OPC UA for Process Automation Devices – PA-DIM™, published by the OPC Foundation; refer to: https://reference.opcfoundation.org/v104/PADIM/v100/docs/

1.5 Terminology

The key words “shall”, “shall not”, and “should” are used in the O-PAS Standard as defined below. These definitions are taken from IETF RFC 2119.

Shall	Definition is an absolute requirement of the specification.
Shall not	Definition is an absolute prohibition of the specification.
Should	There may exist valid reasons in particular circumstances to ignore a particular item, but the full implications must be understood and carefully weighed before choosing a different course.

1.6 Future Directions (Informative)

A future version of the O-PAS Standard will update Part 6.2 to reflect and leverage future changes to the standards (Normative References) listed in Section 1.4. The definition of the configuration of security related to basic configurations will be provided in a future version.

A future version might also address storing Global Discovery Server (GDS) configuration information in a standardized format in the import/export file which currently must be done manually by the user with vendor-specific tools.

A future version of the O-PAS Standard will address a standardized mechanism to download configurations to DCNs.

Also, refer to Appendix C for information regarding future extension to *IOServiceEngineType*.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

2 Terms and Definitions

For the purposes of the O-PAS Standard, the following terms and definitions apply. Merriam-Webster's Collegiate Dictionary should be referenced for terms not defined in this chapter. The OPAF Glossary and Abbreviations should be referenced for additional terms not defined in this chapter.

2.1 OPAF Glossary and Abbreviations

For the OPAF Glossary and Abbreviations, see the definitions at: www.opengroup.org/open-process-automation-forum/glossary.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

3 Information Model

3.1 Information Model

A system made up of O-PAS conformant components has its interfaces defined through multiple information and exchange models. Part 6.2 contains definitions for the common models that can be extended in subsequent subparts of Part 6. Figure 1 illustrates the O-PAS common objects defined in the O-PAS Standard.

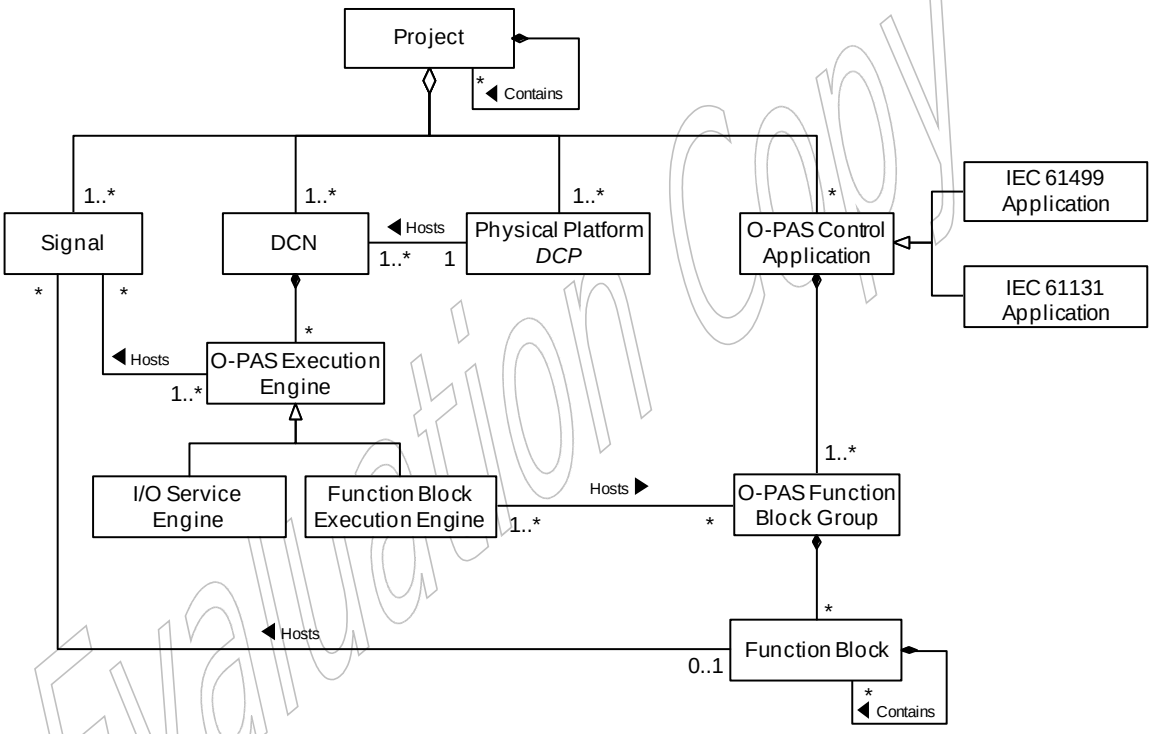


Figure 1: O-PAS Common Objects

Table 1 lists the common objects defined in Part 6.2 and their descriptions. The detailed attributes of these objects are defined in Chapter 4 and Chapter 6.

This O-PAS common object UML model is the base for creating the Automation Markup Language (AutomationML or AML) configuration file as well as the OPC UA runtime model for a DCN.

The UML model contains two branches that lead to the function blocks; one focused on describing on which execution engine a function block executed (DCN – Function Block Execution Engine – O-PAS Function Block Group – Function Block), and another focused on the logical arrangement of function blocks in applications (O-PAS Control Application – O-PAS Function Block Group – Function Block).

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

The AML configuration file reflects both branches.

The OPC UA runtime model of a DCN is a subset of the UML model. The branch including the O-PAS control application is not currently represented in the runtime model.

Table 1: Common Objects and Definitions

Object	Description
Project	A collection of related objects associated with a specific configuration of hardware, software, and Layer-F applications.
Distributed Control Platform (DCP)	The hardware that runs the project's software and applications. Also identified as a physical platform. A project contains one or more physical platforms. A physical platform hosts zero or more DCNs. (Note: In the development phase there may be definitions of physical platforms that have not been assigned DCNs.)
Distributed Control Node (DCN)	The software environment for execution of all layers of applications. See O-PAS Part 1 – Technical Architecture Overview (Informative) for the definition. A project references one or more DCNs. A DCN is hosted in a physical platform. (Note: In the development phase there may be definitions of DCNs that have not been assigned to physical platforms.)
O-PAS Engine	The O-PAS Standard identifies certain types of Layer-I applications as engines to signify their functionality. Example 1: Function Block Execution Engine – an O-PAS application that executes an O-PAS Layer-F function-block application. Defined as abstract in O-PAS Part 6.2; extended by O-PAS Part 6.5 and Part 6.6. Example 2: Input/Output Service Engine – an O-PAS application that exposes its data via the OCF. Defined in O-PAS Part 6.2. Example 3: Alarm Engine – an application within a DCN that processes O-PAS alarms. Defined in O-PAS Part 6.3.
Signal	Representation of a physical quantity or calculated value.
O-PAS Control Application	An abstract class that represents one of the various types of Layer-F applications defined in the O-PAS Standard.
O-PAS Function Block Group	Collection of Layer-F function blocks. All O-PAS function blocks in an O-PAS function block group belong to the same O-PAS control application and are hosted by the same O-PAS engine.
Function Block	An abstract class as defined in Section 3.3 below. The attributes of a function block are defined in Chapter 4.
IEC 61499 Application	An O-PAS control application that is programmed using the programming languages and practices defined in IEC 61499.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Object	Description
IEC 61131 Application	An O-PAS control application that is programmed using the programming languages and practices defined in IEC 61131.

3.2 O-PAS Signals

The O-PAS Standard employs data types compatible with the signals defined in the OPC UA Process Automation Device Information Model (PA-DIM™) standard. The PA-DIM standard, jointly developed by the OPC Foundation and the FieldComm Group, represents the information that is available to applications, Human-Machine Interfaces (HMIs), and other systems from process automation devices.

OPC variables exchanged over OCF are termed O-PAS signals. O-PAS signals are in the same type hierarchy as PA-DIM signals, and anywhere an O-PAS signal is required, a PA-DIM signal may be used.

O-PAS signals are used to exchange field values, timestamp and status:

- To and from the I/O modules, or
- Between O-PAS components

All field values, along with their status and timestamp, are provided as O-PAS conformant signals. Figure 2 is a UML illustration of the signal model defined in the PA-DIM standard.

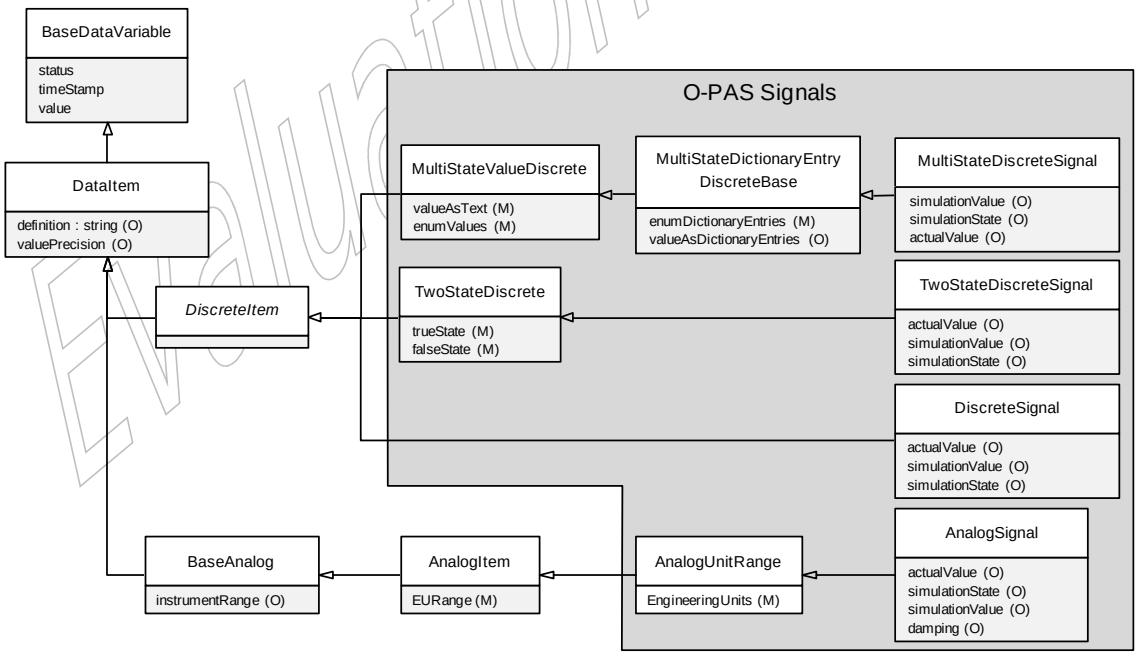


Figure 2: O-PAS UML Signal Data Type Model

Figure 2 illustrates the UML diagram of the O-PAS signal data type used by the O-PAS Standard, and Figure 3 is the equivalent OPC UA model of the same *SignalTypes*. An O-PAS signal is based on one of the types highlighted in the grey box labeled "O-PAS Signals" in the

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

figures or its sub-types. The parent types, shown outside the grey box, are not to be used as O-PAS signals. The O-PAS signal shall have an OPC UA alias in the *TagVariable* alias, defined in OPC UA Part 17 category pointing to it [CRQ-P6.2-0002].

OPC UA variables representing O-PAS signals are illustrated in Figure 3 and also listed in Table 2. Vendors shall use the *SignalTypes* defined in Table 2 or types derived from those types [CRQ-P6.2-0003]. These restrictions are needed to ensure interoperability.

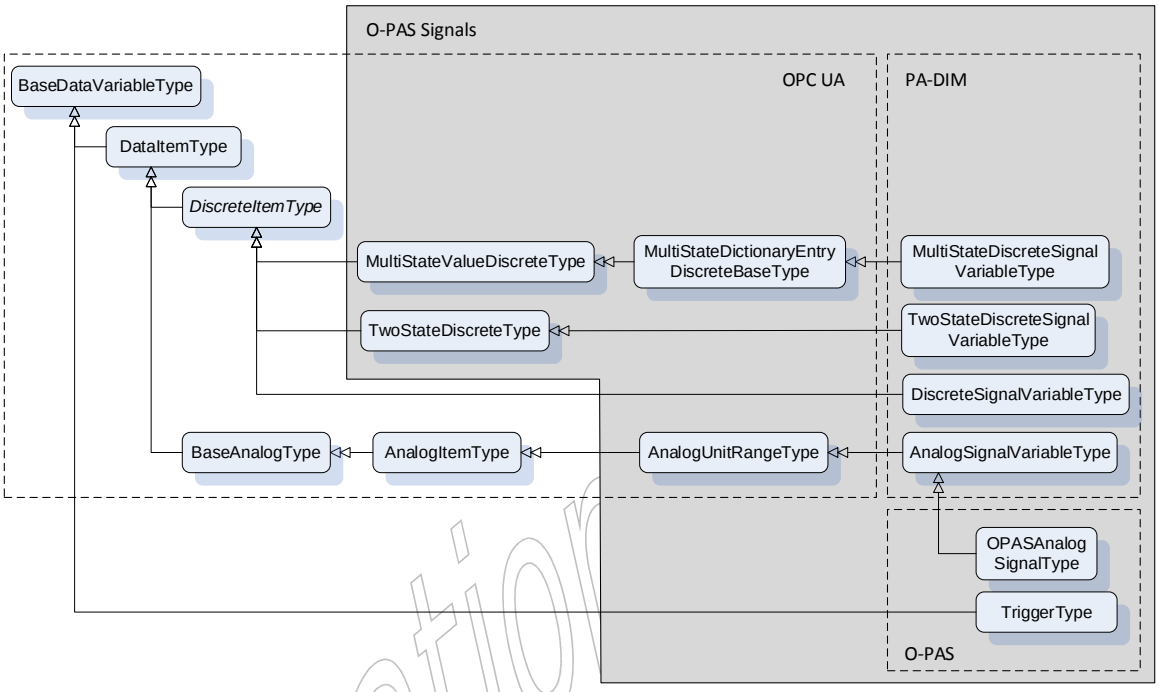


Figure 3: O-PAS Signals

For the discovery of signals across the OCF, the O-PAS Standard utilizes OPC UA aliases as defined in OPC UA Part 17.

Table 2: OPC UA Base Types for O-PAS Signals

VariableType	Defined By
AnalogUnitRangeType	OPC Foundation
TwoStateDiscreteType	OPC Foundation
MultiStateValueDiscreteType	OPC Foundation
MultiStateDictionaryEntryDiscreteBaseType	OPC Foundation
AnalogSignalVariableType	PA-DIM Standard
DiscreteSignalVariableType	PA-DIM Standard
TwoStateDiscreteSignalVariableType	PA-DIM Standard

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

VariableType	Defined By
MultiStateDiscreteSignalVariableType	PA-DIM Standard
OPASAnalogSignalType	O-PAS Standard
TriggerType	O-PAS Standard

The O-PAS Standard expects that within each of these types there are subsets which are available to a control language. These type subsets are available and used in AML for offline engineering and are also available on the OCF for transmission.

An O-PAS Connectivity Function Block (OCFB) links an O-PAS signal to a data or trigger input or output of an O-PAS function block. O-PAS signals are used in connecting control schemes among DCNs. OCFBs are implemented in a function block programming language. For example, IEC 61499 could use composite function blocks that could contain Service Interface Function Blocks (SIFBs) and basic function blocks.

OCFBs have two defined uses:

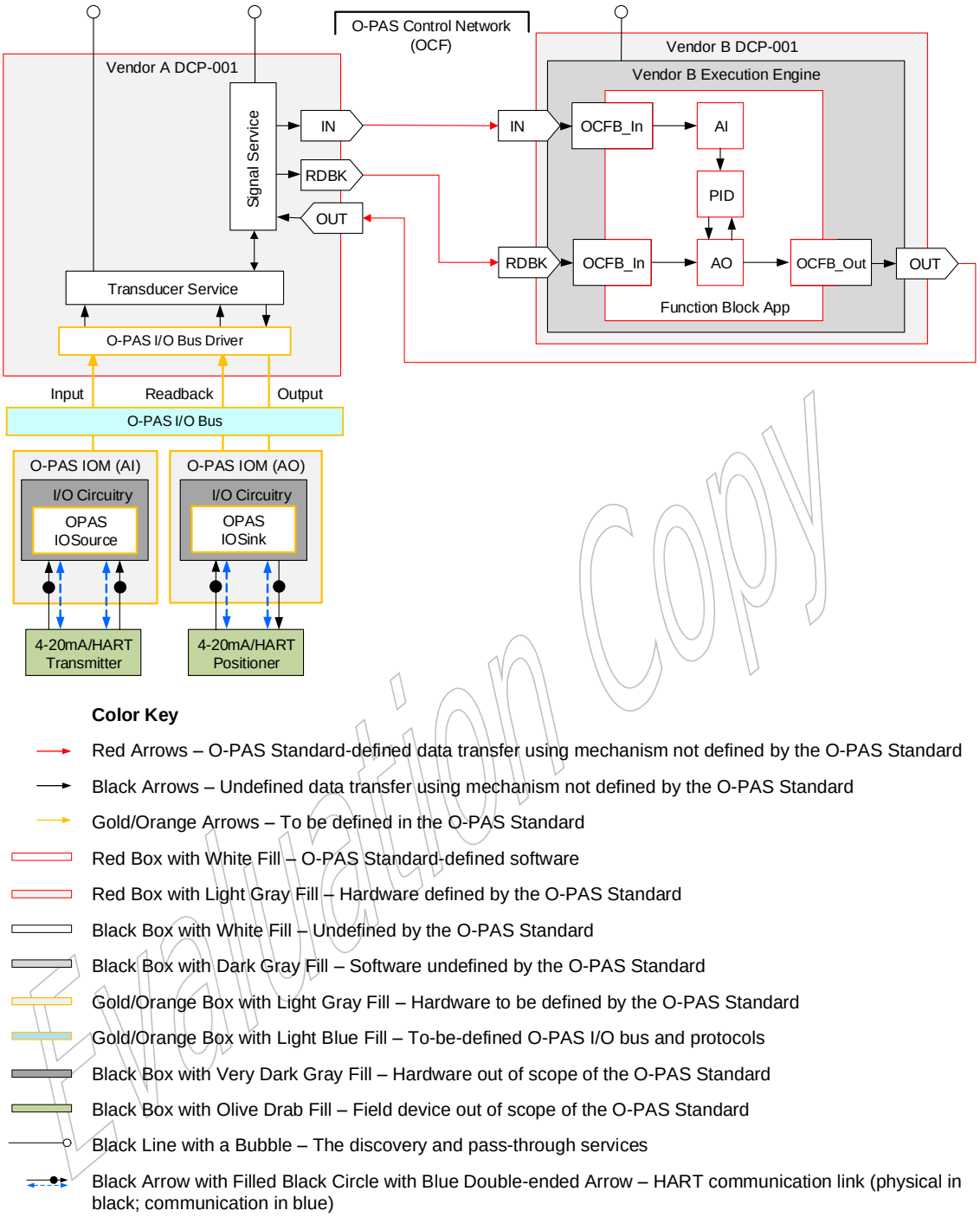
1. Bringing an O-PAS signal into a function block execution engine
2. Sending an O-PAS signal out of a function block execution engine

While some function block implementations, like those based on IEC 61499, provide a peer-to-peer communication mechanism, it is not standardized. The O-PAS Standard provides a standard communication mechanism to assure communication interoperability. At a general level, it specifies the use of OPC UA and the O-PAS Information Model. More specifically, it uses the OCFB to normalize all control-related communication between function block execution engines and other O-PAS engines.

Figure 4 illustrates an example of O-PAS components that might be combined to create a working control application. The following requirements apply to Figure 4:

- The vendor-specific I/O service, which consists of the Signal Service and Transducer Service, makes each of its input values available as OPC UA signals using a configured OPC UA alias for each input
- The vendor-specific I/O service makes each of its output values available as an OPC UA signal; it finds its source signal by using its configured O-PAS signal alias or allows manual changes via the OPC UA if not configured to a specific alias
- The OCFB_IN, defined in Section 6.4.13, finds its input by using its configured O-PAS signal alias or allows manual changes via the OPC UA if not configured to a specific alias
- The OCFB_OUT, defined in Section 6.4.12, makes its value available to the OPC UA as an O-PAS signal using the configured alias
- A vendor-specific I/O subsystem and the vendor side of the OCFBs may optimize operation such that a search of the outside world is not required if the alias is local

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution



Color Key

- Red Arrows – O-PAS Standard-defined data transfer using mechanism not defined by the O-PAS Standard
- Black Arrows – Undefined data transfer using mechanism not defined by the O-PAS Standard
- Gold/Orange Arrows – To be defined in the O-PAS Standard
- Red Box with White Fill – O-PAS Standard-defined software
- Red Box with Light Gray Fill – Hardware defined by the O-PAS Standard
- Black Box with White Fill – Undefined by the O-PAS Standard
- Black Box with Dark Gray Fill – Software undefined by the O-PAS Standard
- Gold/Orange Box with Light Gray Fill – Hardware to be defined by the O-PAS Standard
- Gold/Orange Box with Light Blue Fill – To-be-defined O-PAS I/O bus and protocols
- Black Box with Very Dark Gray Fill – Hardware out of scope of the O-PAS Standard
- Black Box with Olive Drab Fill – Field device out of scope of the O-PAS Standard
- Black Line with a Bubble – The discovery and pass-through services
- Black Arrow with Filled Black Circle with Blue Double-ended Arrow – HART communication link (physical in black; communication in blue)

Figure 4: Example DCN with Embedded I/O Feeding DCN with Embedded Control and No I/O

3.3 O-PAS Function Block

The core of an O-PAS Standard-conformant control application consists of a set of function blocks, each performing useful calculation(s). The classic example is a “PID loop” which is normally represented by a block that handles the input signal (scaling, characterization, etc.), a block to execute the control algorithm, and a block to drive the output to the field.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

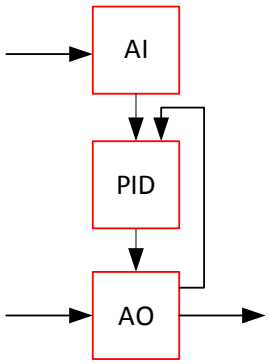


Figure 5: A PID Loop

In the example illustrated in Figure 5, we have three O-PAS specified function blocks. The Analog Input (AI) function block receives the value from a sensor or other source. The PID function block performs the calculation and takes feedback from the Analog Output (AO) function block. The AO function block receives a readback signal from the field while also generating the field value. The PID function block will know through its feedback link if there are problems.

An O-PAS function block defined in this document is intended to be a base abstract class that can be used in the subsequent subparts of O-PAS Part 6, as the superclass for O-PAS Layer-F application function blocks. It has the characteristics and behavior set out in Figure 6.

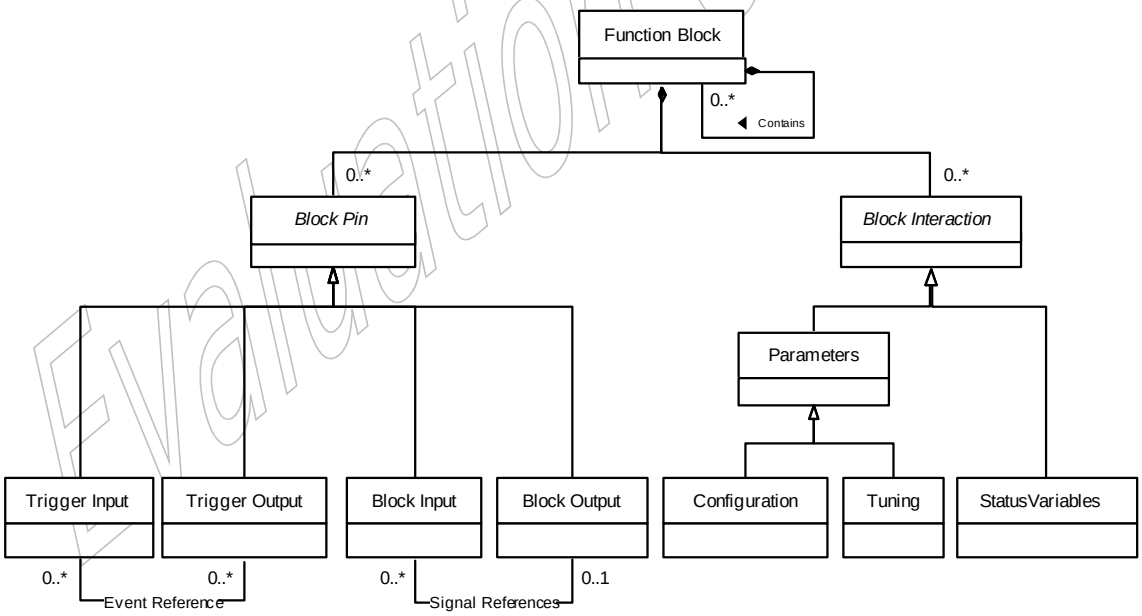


Figure 6: O-PAS Function Block Model

Descriptions for O-PAS common objects are listed in Table 3.

Table 3: O-PAS Common Objects

Object	Description

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Object	Description
Function Block Pin	These are the function block inputs and outputs (data and triggers).
Function Block	Function blocks describe the function between input and output variables. Inputs and outputs of blocks are connected together via connections or lines in the programming environment.
Trigger Input	A connectable input to the function block which triggers an action in the function block. It may be connected to a trigger output of a function block. For example, an IEC 61499 event or a timer signal that triggers a cyclic function block.
Trigger Output	An output of a function block which may be a trigger input to one or more trigger inputs. For example, an IEC 61499 event.
Event Reference	Defined association between a trigger input and output.
Data Input	A connectable data input to a function block. It may be connected and, if so, it may be connected to a block data output of a function block.
Data Output	A data output from a function block which may be connected to one or more block data inputs.
Signal Reference	Defined association between block input and output.
Function Block Interaction	Represents an interface that the OPC UA server can expose the function block parameters including configuration parameters, tuning parameters, and status variables that represent internally visible information within a block. The intention of the interface is to provide data for monitoring, operation, and configuration of the function block to OPC UA clients.
Parameters	Function block interaction parameters. Intended for operation and configuration.
Configuration Parameter	Represents function block data which is intended to be used by O-PAS applications. It is not intended to be connected to another function block. Changes to configuration parameters might be dependent on the operational state of the function block and user access role.
Tuning Parameter	A parameter representing function block data which is intended to be used by O-PAS applications. It is not intended to be connected to another function block. It may be changed when the block is executing. The ability to change may be dependent on the operational state of the function block and user access role.
Status Variable	A variable that is available from the function block. It represents function block data which is intended to be monitored by O-PAS applications. It is not intended to be connected to another function block. An example of a status variable is block error.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

3.4 Basic Model Configuration Constraints

There are no configuration constraints on any of the referenced standards.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

4 Import/Export Format

4.1 Import/Export Format Structure

This chapter defines the import/export format for the O-PAS basic configuration. The import/export format uses the Open Packaging Conventions (ISO/IEC 29500-2) file (.opas) described in O-PAS Part 6.1 – Information and Exchange Models: Overview and Interfaces containing an AML (IEC 62714-1) file. This section describes how the .opas file format is used for the exchange of configuration information and Layer-F applications between vendor tools.

This document focuses on basic configuration which will be stored in the AML document. This chapter describes how the different AML pieces are used. For attributes and formal definitions, see Appendix A.

4.2 O-PAS Common Objects

Figure 7 shows some of the top-level role classes when the O-PAS common objects described in Section 3.1 are mapped to AML. Details on the various pieces can be found in the following sections. In Figure 7, the boxes in blue represent the different AML role classes.

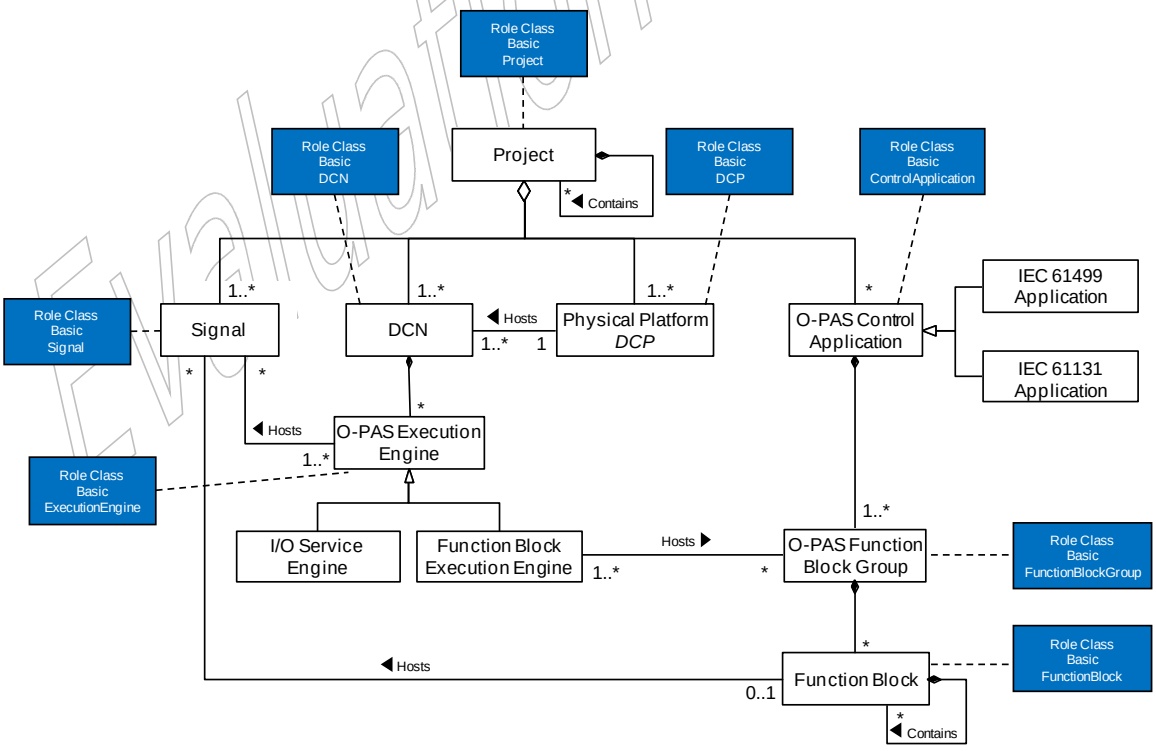


Figure 7: Mapping O-PAS Common Object UML to AML

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

4.2.1 Basic-Project Role Class

The top-level element in the InstanceHierarchy. It is the AML representation of the "Project" in Section 3.1 and bundles elements associated with a specific configuration of hardware, software, and Layer-F applications.

4.2.2 UA-Tag Role Class

The elements in the InstanceHierarchy that have an OPC UA alias associated with them. The primary purpose is to serve as the parent role for other role classes that reflect specific types of aliases; e.g., *UA-TagVariable*.

It has an attribute named "*AliasName*" of type *UA-AliasName* to store the string for the UA alias.

4.2.3 UA-TagVariable Role Class

Inherits from *UA-Tag* and is used for the elements in the InstanceHierarchy that have an OPC UA alias associated with them that falls into the *TagVariable* category; i.e., points to a UA variable.

It contains an external interface named "*UAConnection*" of type *UA-VariableConnection* with Direction "Out". This interface is used to connect non-control consumers such as HMIs, alarms, and historians to this variable.

4.2.4 Basic-Signal Role Class

Inherits from *UA-TagVariable* and is used for the elements in the InstanceHierarchy that represent O-PAS signals as defined in Section 3.2. This role class is the AML representation of the "Signal" in Section 3.1.

It contains an external interface named "*SignalInterface*" of type *Basic-SignalInterface* with Direction "InOut". This Interface is used to form an internal link to the producer and consumers of the signal.

4.2.5 Basic-SignalInterface Interface Class

Used in elements that deal with O-PAS signals, such as *Basic-Signal* in the *Tag* list, InternalElements in *Basic-IOServiceEngine* that represent individual I/O channels that provide/consume O-PAS signals, or OCFBs that provide/consume O-PAS signals.

This interface class has a Direction attribute to indicate the direction of the OCF information flow as seen from the field device. For example, a sensor in the field is an "Out" because it puts a signal out to the OCF. Similarly, an actuator is an "In" because it takes a signal in from the OCF.

Internal links from this interface class will only link to other instances of this interface class. The normal linking goes from an interface with Direction "Out" (the producer of the signal) to another interface with Direction "InOut" that is part of a *Basic-Signal* element in the *Tag* list. From there it connects to one or many interfaces with Direction "In" (consumers of the signal).

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

4.2.6 Basic-TriggerInterface

Inherits from *Basic-SignalInterface* and is used for O-PAS signals that are triggers.

4.2.7 Basic-TagList Role Class

Provides the *Tag* list for a project. It groups all the elements with the *UA-Tag* role together to provide an easy overview of the OPC UA aliases used in the project.

4.2.8 Basic-VendorNameplate Attribute Type

Stores the vendor nameplate information of the OPC UA interface *IVendorNameplateType* defined in OPC UA Part 100.

It can be used on AML elements that represent UA objects which implement the interface.

4.2.9 Basic-TagNameplate Attribute Type

The attribute type *Basic-TagNameplate* is used to store the *Tag* nameplate information of the OPC UA interface *ITagNameplateType* defined in OPC UA Part 100.

It can be used on AML elements that represent UA objects which implement the interface.

4.2.10 Basic-DCN Role Class

The AML representation of the "DCN" in Section 3.1. It contains the configuration attributes for a DCN and serves as the parent element for the execution engines in the DCN.

A vendor would usually create a System Unit Class (SUC) with *Basic-DCN* as a role requirement for their DCN product. Instances of such an SUC should be created as InternalElements to a *Project-Role* element in the InstanceHierarchy.

Since DCNs have OPC UA objects associated with them, they are used as the root of a hierarchy of AML elements whose relative relationship reflects the relative relationship of OPC UA nodes within the DCN UA object. See *UA-NodeRelative* in O-PAS Part 6.1 for details.

4.2.11 Basic-DCP Role Class

The AML representation of the "DCP" in Section 3.1. It contains the configuration attributes for a DCP and provides a hierarchy for the DCNs it contains.

Since DCN elements are already InternalElements under a *Project-Role* element, the "Hosts" relationship between the DCP and DCN is expressed by creating mirrors for the DCN elements inside the DCP element. See Figure 8 for an example.

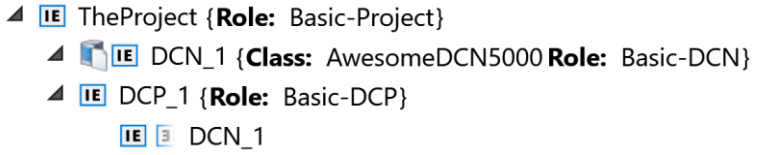


Figure 8: Example of Using Mirrors to Express the DCP Hosting DCNs Relationship in AML

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Note: "*AwesomeDCN5000*" is an SUC created for the DCN model "*AwesomeDCN5000*" sold by a vendor.

4.2.12 Basic-ControlApplication Role Class

AML representation of the "Control Application" in Section 3.1. For specialized kinds of control applications – e.g., such that are based on function blocks – dedicated role classes that inherit from *Basic-ControlApplication* should be created; e.g., *Basic-FunctionBlockApplication* for function block-based applications.

This role class includes an external interface of class *Basic-CodeFileRelationshipInterface* called "*LogicSource*" that shall be used to link to the source code file containing the control logic definition for the application.

4.2.13 Basic-FunctionBlockApplication Role Class

Inherits from *Basic-ControlApplication*. It serves as the parent element for and groups together function block groups that form control applications that provide process control.

4.2.14 Basic-BaseEngine Role Class

The AML representation of the "Execution Engine" in Section 3.1. It contains the configuration attributes for an execution engine.

Usually, each execution engine in a DCN has an OPC UA object that is a component of a *DCNBase* type object representing the DCN hosting the execution engine. This means that the OPC UA location of the execution engine relative to the *DCNBase* object is usually known, and thus elements in the InstanceHierarchy that have the role requirement *Basic-ExecutionEngine* role class should also have the role requirement *UA-NodeRelative* class defined in O-PAS Part 6.1.

A vendor would usually create an SUC with *Basic-ExecutionEngine* (or one of the standard child-roles of it defined below) as a role requirement for their execution engine product. Instances of such an SUC should be created as InternalElements to a *DCN-Role* element in the InstanceHierarchy.

In AML, function block groups already are InternalElements of elements with the role *Basic-ControlApplication*. Therefore, the "Hosts" relationship between the execution engine and function block groups is expressed by creating mirrors for the function block groups in the execution engine element, similar to the DCP hosts DCN relationship.

4.2.15 Basic-FBExecutionEngine Role Class

Inherits from *Basic-BaseEngine* and is the AML representation of an "Execution Engine" in Section 3.1 that is dedicated to executing function block-based applications such as IEC 61131-3 or IEC 61499. It contains the configuration attributes for a function block execution engine and serves as the parent element for the function block groups hosted by/deployed to the execution engine.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

4.2.16 Basic-SignalService Role Class

The AML representation of an "*ISignalServiceType*" in Section 6.4.4. It is used to mark elements that contain other elements which represent O-PAS signals and have *Basic-SignalInterface* interfaces.

4.2.17 Basic-IOServiceEngine Role Class

Inherits from *Basic-BaseEngine* and is the AML representation of an "Execution Engine" in Section 3.1 that is dedicated to providing I/O data to the OCF. For details on I/O, see O-PAS Part 7.

The role class contains the configuration attributes for an I/O service engine and serves as the parent element for InternalElements representing the different I/O-Channels. These InternalElements should contain *Basic-SignalInterface* interfaces with Direction "In" or "Out".

SUCs or elements in the InstanceHierarchy that have this role as a role requirement shall also have *Basic-SignalService* as a role requirement.

4.2.18 Basic-FunctionBlockGroup Role Class

The AML representation of the "Function Block Group" in Section 3.1. It contains the configuration attributes for a function block group and serves as the parent element for the function blocks (instances) that make up the function block group.

Usually, each function block group in an execution engine has an OPC UA object that is a component of an *ExecutionEngine* type UA object representing the execution engine hosting the function block group. This means that the OPC UA location of the function block group relative to the execution engine object is usually known, and thus elements in the InstanceHierarchy that have the role requirement *Basic-FunctionBlockGroup* role class should also have the role requirement *UA-NodeRelative* class defined in O-PAS Part 6.1.

This role class includes an external interface of class *Basic-DownloadFileRelationshipInterface* called "*DownloadFile*" that is used to link to the download/upload file (see Chapter 5) for this particular function block group. The file referenced with this relationship is what is sent to the execution engine to provide it with all the information needed to execute the function block group.

4.2.19 Basic-FunctionBlock Role Class

Inherits from *UA-TypeDefinition* and is the AML representation of the "Function Block" in Section 3.1. It contains the configuration attributes for a function block. It can contain InternalElements for other function blocks as well as InternalElements for variables in the block (code and/or UA ObjectType) that are needed in the AML.

Function blocks are typically defined as types that are then instantiated multiple times. For each function block type defined by a vendor or the user themselves, an SUC with the role requirement *Basic-FunctionBlock* shall be created. These SUCs shall contain instances for all function blocks instantiated inside the block. The SUC is instantiated in the InstanceHierarchy to create an instance of the function block. O-PAS Part 6.4 defines SUCs for the O-PAS specified set of standard function blocks. These SUCs can be used as examples of how to define SUCs for a function block type.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

SUCs for a function block need to be linked to a UA ObjectType definition that is used in the OPC UA server of the execution engine to create UA objects for each function block instance. The link to the UA ObjectType uses the external interface "*UATypeLinkInterface*" of class *UA-NodeInterface* that is part of the parent role class, *UA-TypeDefinition*.

The *Basic-FunctionBlock* role class should only be used on SUCs, not directly on any elements in the InstanceHierarchy.

In addition, the *Basic-FunctionBlock* role class has an external interface called "*LogicSource*". This interface is used to link to the file containing the definition of the logic of the block; e.g., an IEC 61131-10 file for an IEC 61131-3 function block. The link is to the file only, not a particular element in the file.

For a better understanding of how function blocks can utilize a combination of AML, OPC UA nodeset, and code files, see O-PAS Part 6.1, Appendix F.

Usually, each function block, which can be contained by either a function block group or another function block, has an OPC UA object that is a component of a function block group or function block type UA object representing the containing the function block (group). This means that the OPC UA location of the function block relative to the containing function block (group) object is usually known, and thus elements in the InstanceHierarchy that have the role requirement *Basic-FunctionBlock* should also have the role requirement *UA-NodeRelative* class defined in O-PAS Part 6.1.

4.2.20 Basic-OCFB_IN System Unit Class

Represents function blocks that take on the OCF and make them available inside a function block engine. It contains an external interface of class *Basic-SignalInterface* (or a child thereof) called "Signal" with Direction "Out".

The SUC has the following role requirements:

- *Basic-FunctionBlock*
- *Basic-SignalService*

4.2.21 Basic-OCFB_OUT System Unit Class

Represents function blocks that take variables internal to a function block engine and make them available on the OCF. It contains an external interface of class *Basic-SignalInterface* (or a child thereof) called "Signal" with Direction "In".

The SUC has the following role requirements:

- *Basic-FunctionBlock*
- *Basic-SignalService*

4.2.22 UA-UOMLibrary Role Class

Collects project-specific *UA-UOMNamespaces*. The units of measure are used to provide UA servers with information needed to display the units properly.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

4.2.23 UA-UOMNamespace Role Class

Can be used to define custom units of measure namespaces with a URI and all units (including *UnitId*, localized name, and description). Instances of this role contain InternalElements with the role requirement *UA-UnitOfMeasure*. The OPC UA server for an application can utilize the *NamespaceUri* and *UnitId* to look up in the *UA-UOMLibrary* of the project which units exist and display them with full localized name and description.

4.2.24 UA-UnitOfMeasure Role Class

Can be used to store the information needed to use units of measure in code (*UnitId*) and display them properly on an HMI (localized name and description).

In has an interface of class *UA-UOMLink* named "*UOMLink*" with Direction "Out" that is used to link to elements using the unit of measure.

4.2.25 UA-UOMLink Interface Class

This interface is used to link units of measure to instances in the InstanceHierarchy that utilize this unit of measure.

This interface class has a Direction attribute to indicate the direction of the unit of measure assignment. All instances of this interface contained in *UA-UnitOfMeasure* have Direction "Out", while instances in elements that are the targets of unit of measure assignments have Direction "InOut". Internal links between "Out" and "InOut" or "InOut" and "InOut" indicate the connections. All chains of internal links must contain exactly one interface with Direction "Out".

4.3 Import/Export Profile

4.3.1 OBC-201: Basic Configuration Import/Export Profile

3. A configuration editing tool shall export an application configuration to an AML file that uses the AML types and classes defined in Appendix A [CRQ-P6.2-0004].
4. A configuration editing tool shall import any AML file that uses the AML types and classes defined in Appendix A [CRQ-P6.2-0005].
5. A configuration editing tool shall assign the *Basic-Project* role to any project information exported in the configuration file [CRQ-P6.2-0006].
6. A configuration editing tool shall assign the *Basic-DCN* role to any DCN information exported in the configuration file [CRQ-P6.2-0007].
7. A configuration editing tool shall implement O-PAS package file [OBC-F-101] Facet from O-PAS Part 6.1 [CRQ-P6.2-0008].

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

5 Download/Upload

For Version 2.1 of the O-PAS Standard, the download/upload exchange file formats are defined by the vendors' configuration editing or configuration management tools to match their DCN software.

For Version 2.1 conformant components, vendor tools would use the import/export format for the O-PAS basic configuration information defined in Chapter 4 to generate their vendor-specific download/upload format files.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

6 OCF Information Model

6.1 O-PAS Information Model

The combination OPC UA information models defined in all parts of the O-PAS Standard make up the O-PAS Information Model.

Figure 9 shows a definition of the O-PAS *DCNBaseType*, a sub-class derived from the OPC UA *ComponentType*. An O-PAS conformant instance of a DCN shall derive from *ComponentType* and implements the *IDCNBaseType* interface [CRQ-P6.2-0010]. The model enables flexibility with inclusion of existing and future device types. For example, any sub-types of *PADIMType* (defined in the PA-DIM standard) or *DeviceType* (defined in OPC UA Part 100) that implements the *IDCNBaseType* interface are O-PAS conformant. *DCNBaseType* is formally defined in Section 6.2.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

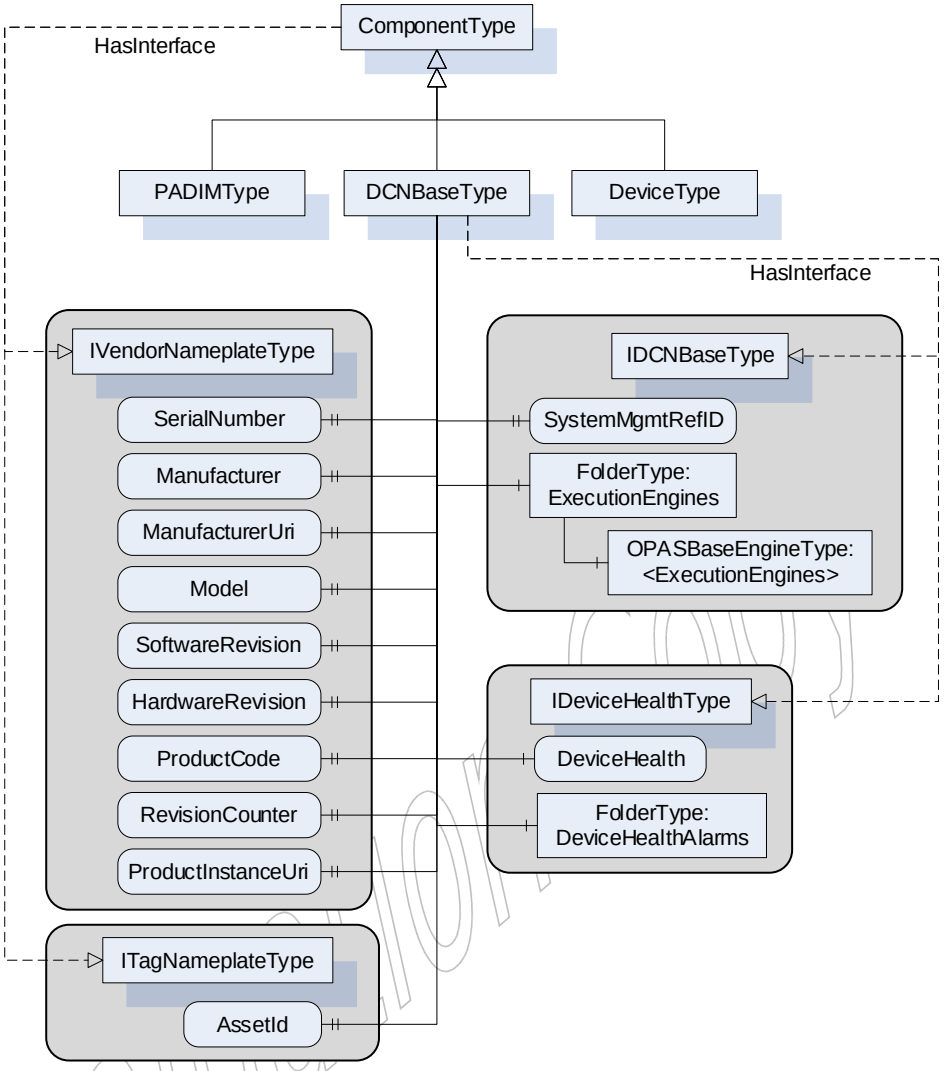


Figure 9: O-PAS OPC UA DCNBaseType

The *OPASBaseEngineType* is an abstract type derived from OPC UA *BaseObjectType* as illustrated in Figure 10. The *IOServiceEngineType* and *FBExecutionEngineType* are both derived from the *OPASBaseEngineType*. O-PAS conformant I/O DCNs shall implement the *IOServiceEngineType* or a sub-type of it [CRQ-P6.2-0009]. Any O-PAS Layer-I application that hosts O-PAS function block(s), such as IEC 61131-3 or IEC 61499, implements the *FBExecutionEngineType* or a sub-type of it. The *AlarmEngineType* is defined in O-PAS Part 6.3. As illustrated in Figure 9, a DCN may contain one or many instances of a sub-type of *OPASBaseEngineType*.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

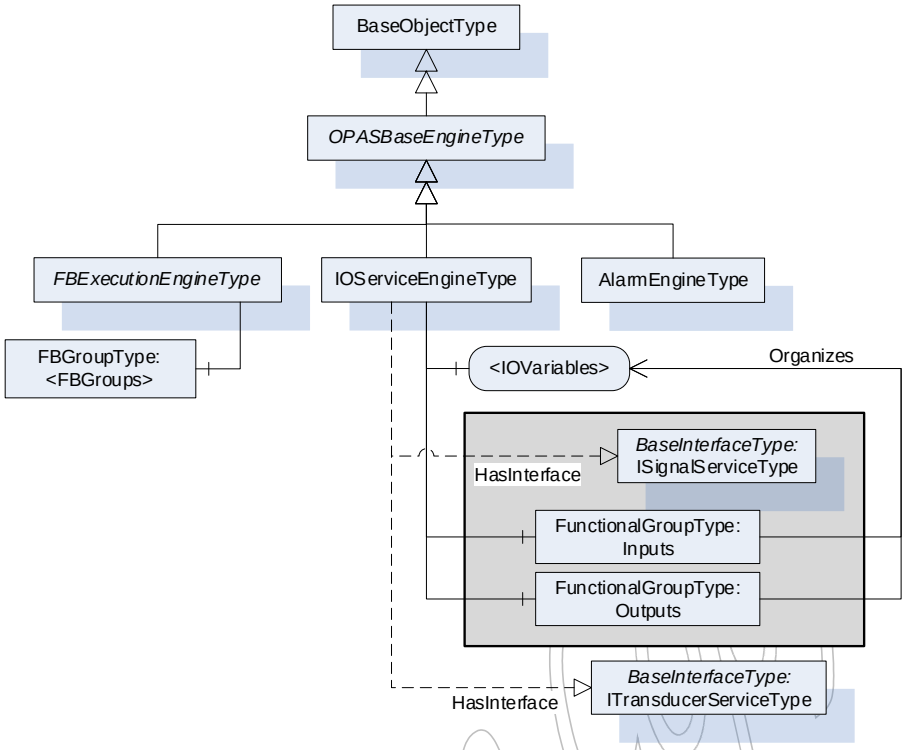


Figure 10: O-PAS Engine Types

In this version of the O-PAS Standard, the *IOServiceEngineType* definition has only one interface; that is, the *ISignalServiceType* as illustrated in Figure 10. Appendix C shows the OPAF intention to define additional extensions of the *IOServiceEngineType* in a future version. All types in the figures are formally defined in Section 6.4.

6.2 OPC UA Information Model PA-DIM Overview

The O-PAS Standard leverages the OPC UA PA-DIM standard for process signals, as illustrated in Figure 11. A DCN is not a PA-DIM device, but it connects to PA-DIM devices for I/O signals. The PA-DIM standard defines a base device concept and information model, which is intended to be used for process automation devices. The model also exposes IEC 61987 semantic dictionary IDs.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

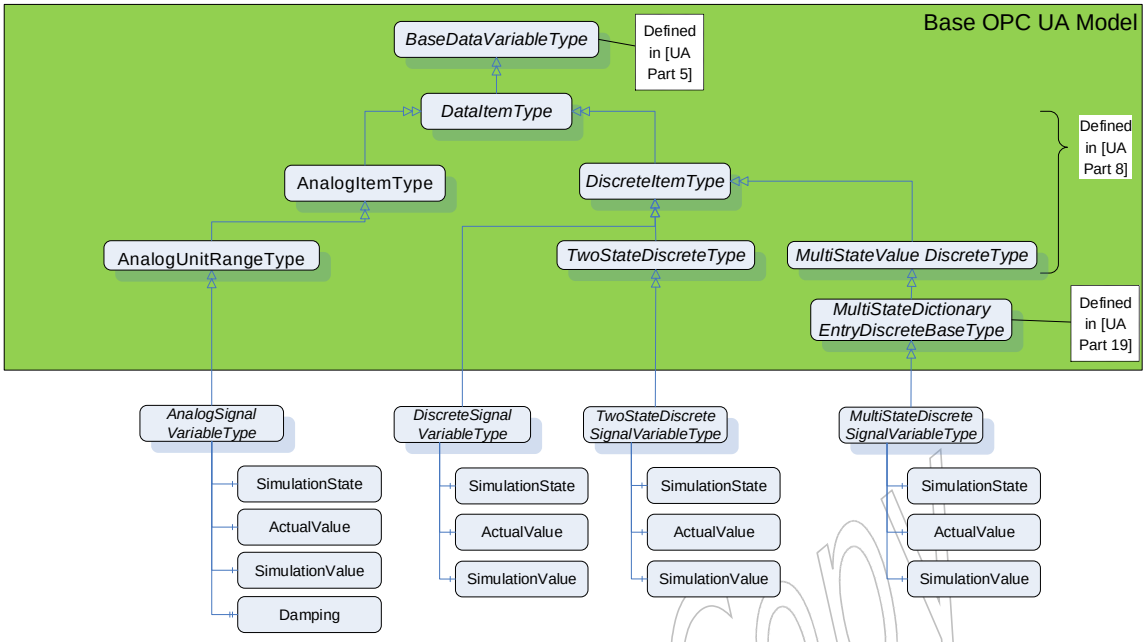


Figure 11: OPC UA PA-DIM SignalVariableType

The *AnalogSignalVariableType* defined in the PA-DIM standard can be used to model any analog *SignalType*. It extends the base *AnalogUnitRangeType* defined in OPC UA Part 8 to add additional information such as values for simulation and damping. Refer to the PA-DIM standard for more details and a definition of the *AnalogSignalVariableType*.

6.3 O-PAS OPC UA Function Block Information Model

Figure 12 shows the O-PAS OPC UA Information Model used to represent the information in a function block that is visible through the OCF. All types in Figure 12 are formally defined in Section 6.4.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

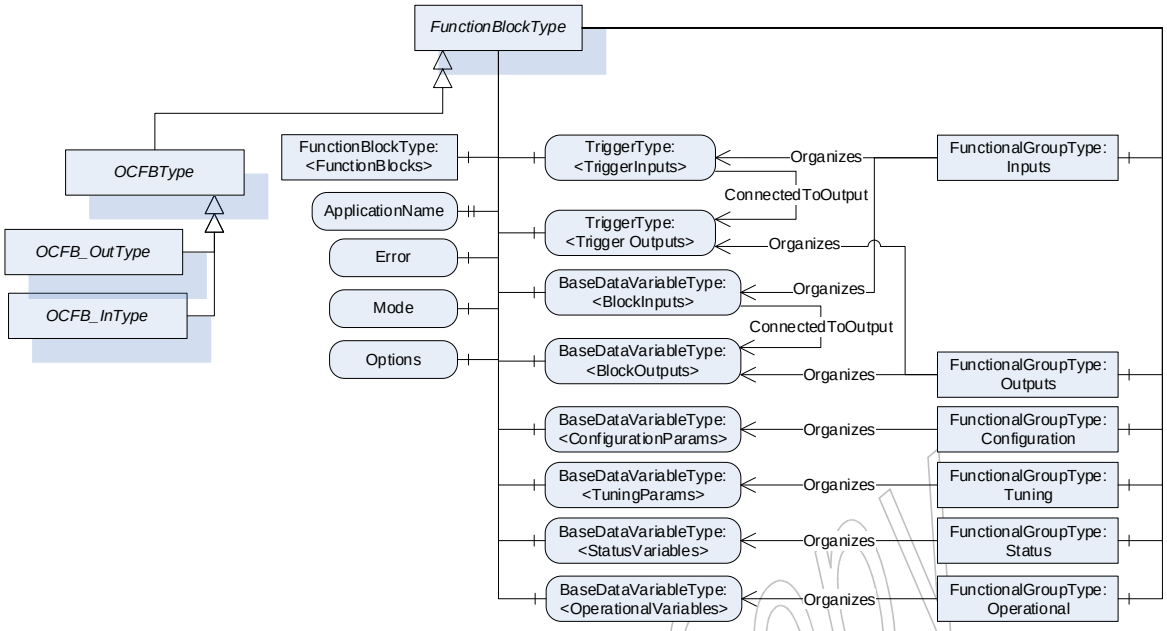


Figure 12: O-PAS OPC UA Function Block Information Model

6.4 O-PAS OPC UA ObjectTypes

The following sections define the O-PAS OPC UA ObjectTypes.[1]

All functionality designated as “M” or “Mandatory” shall be implemented as described to be O-PAS conformant. If a product chooses to implement functionality indicated as “O”, “Optional”, or “OptionalPlaceholder” it shall be implemented to conform to the indicated functionality.

6.4.1 IDCNBaseType

The *IDCNBaseType* provides the interface for O-PAS conformant DCNs to the OCF. The *IDCNBaseType* is derived from OPC UA *BaseInterfaceType* and is formally defined in Table 4.

Table 4: IDCNBaseType Definition

Attribute	**Value**				
BrowseName	IDCNBasetype				
IsAbstract	FALSE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of BaseInterfaceType defined in OPC UA Part 7.					
0:HasProperty	Variable	SystemMgmtRefID	0:String	0:PropertyType	Optional

[1] For definitions of OPC UA base objects, refer to the corresponding standards at https://opcfoundation.org.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

0:HasComponent	Object	ExecutionEngines		0:FolderType	Optional

The *SystemMgmtRefID* is a unique identification string that can be used to identify the system management interface running on the same DCN.

The *ExecutionEngines* folder shall contain instances of *OPASBaseEngineType*, defined in Section 6.4.3, or sub-types thereof [CRQ-P6.2-0011].

6.4.2 DCNBaseType

A DCN is a type of O-PAS conformant component that connects to the OCF. Refer to O-PAS Part 1 for more details on DCNs. *DCNBaseType* is derived from *ComponentType*, defined in OPC UA Part 100, and applies the *IDeviceHealthType* and *IDCNBaseType*. It is formally defined in Table 5.

Table 5: DCNBaseType Definition

Attribute	**Value**				
BrowseName	DCNBaseType				
IsAbstract	FALSE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of 2:ComponentType defined in the OPC UA Part 100.					
HasInterface	ObjectType	IDCNBaseType	Defined in Section 6.4.1.		
HasInterface	ObjectType	IDeviceHealthType	Defined in OPC UA Part 100.		
Applied from IVendorNameplateType defined on ComponentType in OPC UA Part 100.					
0:HasProperty	Variable	2:Manufacturer	0:LocalizedText	0:PropertyType	Mandatory
0:HasProperty	Variable	2:ManufacturerUri	0:String	0:PropertyType	Mandatory
0:HasProperty	Variable	2:Model	0:LocalizedText	0:PropertyType	Mandatory
0:HasProperty	Variable	2:SerialNumber	0:String	0:PropertyType	Mandatory
0:HasProperty	Variable	2:ProductCode	0:String	0:PropertyType	Mandatory
0:HasProperty	Variable	2:HardwareRevision	0:String	0:PropertyType	Mandatory
0:HasProperty	Variable	2:SoftwareRevision	0:String	0:PropertyType	Mandatory
0:HasProperty	Variable	2:RevisionCounter	0:Int32	0:PropertyType	Mandatory

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Attribute	Value				
BrowseName	DCNBaseType				
IsAbstract	FALSE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
0:HasProperty	Variable	2:ProductInstanceUri	0:String	0:PropertyType	Mandatory
Applied from ITagNameplateType defined on ComponentType in OPC UA Part 100.					
0:HasProperty	Variable	2:AssetId	0:String	0:PropertyType	Mandatory
HasProperty	Variable	ComponentName	0:LocalizedText	PropertyType	Mandatory
Applied from IDeviceHealthType defined in OPC UA Part 100.					
0:HasComponent	Variable	2:DeviceHealth	2:DeviceHealthEnumeration	0:BaseDataVariableType	Mandatory
0:HasComponent	Object	2:DeviceHealthAlarms	0:FolderType		Optional
Applied from IDCNBaseType defined in Section 6.4.1.					
0:HasProperty	Variable	SystemMgmtRefID	0:String	0:PropertyType	Optional

6.4.3 OPASBaseEngineType

The *OPASBaseEngineType* is the abstract base type from which the *IOServiceEngineType* and *FBExecutionEngineType* are derived and defined in Sections 6.4.6 and 6.4.6. Future enhancement may include an OPC UA *TemporaryFileTransferType* instance that enables a common configuration download/upload service for instances of *OPASBaseEngineType*.

Table 6: OPASBaseEngineType Definition

Attribute	Value				
BrowseName	OPASBaseEngineType				
IsAbstract	TRUE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of BaseObjectType defined in OPC UA Part 5.					

Control of access to the shared application parameters and objects is handled by the application using the OPC UA *WriteMask*, *AccessLevel*, and *AccessLevelEx* attributes.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

6.4.4 ISignalServiceType

The *ISignalServiceType* is derived from OPC UA *BaseInterfaceType* and is formally defined in Table 7. The *ISignalServiceType* provides the interface to O-PAS signal variables. The functional groups can be implemented to organize components belonging to types to which the *ISignalServiceType* interface is applied (see the definition for *IOServiceEngineType* in Table 8).

Table 7: ISignalServiceType Definition

Attribute	Value				
BrowseName	ISignalServiceType				
IsAbstract	FALSE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of BaseInterfaceType defined in OPC UA Part 7.					
0:HasComponent	Object	Inputs		2:FunctionalGroupType	Mandatory
0:HasComponent	Object	Outputs		2:FunctionalGroupType	Mandatory

6.4.5 IOServiceEngineType

An instance of *IOServiceEngineType* is an execution engine within a DCN that exposes O-PAS conformant inputs and outputs on the OCF. <SignalVariables> is an optional placeholder for the input and output signal variables hosted by the engine. If any instances of <SignalVariables> exist, they shall be organized by the *FunctionalGroupType* inputs and outputs [CRQ-P6.2-0012]. The functional group non-hierarchal references are defined in Table 9.

Table 8: IOServiceEngineType Definition

Attribute	Value				
BrowseName	IOServiceEngineType				
IsAbstract	FALSE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of OPASBaseEngineType.					
0:HasComponent	Variable	<SignalVariables>		0:BaseDataVariableType	OptionalPlaceHolder
0:HasInterface	Object	ISignalServiceType		Defined in Section 6.4.4.	
Applied from ISignalServiceType.					

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Attribute	Value				
BrowseName	IOServiceEngineType				
IsAbstract	FALSE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
0:HasComponent	Object	Inputs		2:FunctionalGroupType	Mandatory
0:HasComponent	Object	Outputs		2:FunctionalGroupType	Mandatory

Table 9: IOServiceEngineType Functional Group Non-Hierarchal References

Source Path	ReferenceType	Is Forward	Target Path
Inputs	Organizes	True	<SignalVariables>
Outputs	Organizes	True	<SignalVariables>

6.4.6 FBGroupType

The *FBGroupType* is a sub-type of *FolderType* defined in OPC UA Part 5 and will contain function blocks. It is formally defined in Table 10.

Table 10: FBGroupType Definition

Attribute	Value				
BrowseName	FBGroupType				
IsAbstract	FALSE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of 0:FolderType defined in OPC UA Part 5.					
0:HasComponent	Object	<FunctionBlocks>		FunctionBlockType	OptionalPlaceHolder

The *FBGroupType* instance shall contain only instances of O-PAS *FunctionBlockType* or sub-type *FunctionBlockType* [CRQ-P6.2-0013].

6.4.7 FBExecutionEngineType

An instance of *FBExecutionEngineType* is an execution engine within a DCN or DCF that executes function block logic and communications. It is possible that more than one instance of *FBExecutionEngineType* is running on a DCN. The *FBExecutionEngineType* defined in this document is an abstract type that can be sub-classed and defined for each control application

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

type defined in subsequent parts of the O-PAS Standard (Part 6.5 – IEC 61499 and Part 6.6 – IEC 61131).

Table 11 details the properties for the O-PAS *FBExecutionEngineType*.

Table 11: FBExecutionEngineType Definition

Attribute	Value				
BrowseName	FBExecutionEngineType				
IsAbstract	TRUE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of OPASBaseEngineType.					
0:HasComponent	Object	<FBGroups>		FBGroupType	OptionalPlaceholder

The <FBGroups> component of *FBExecutionEngineType* is an optional placeholder for groups of O-PAS function block instances.

6.4.8 FunctionBlockType

The *FunctionBlockType* definition is shown in Table 12. The <FunctionBlocks> component of *FunctionBlockType* is an optional placeholder that indicates an O-PAS function block can contain one or more function blocks. This concept is also known as a composite function block in IEC 61499; there is no equivalent concept in IEC 61131 where all Program Organization Units (POUs) can contain function blocks

Variables declared for a *FunctionBlockType* are referenced using generic *HasComponent* references. To differentiate the purpose of the individual variables, there are additional non-hierarchical references from the functional groups to the variables. These indicate the role(s) that a variable takes in the O-PAS function block model. All non-hierarchical references starting at the functional groups must point to variables (or children of variables) in the function block node containing that functional group. The definition of the non-hierarchical references is shown in Table 9. Descriptions for O-PAS functional groups are listed in Table 13.

A product implementing *FunctionBlockType* shall include at least one of the "OptionalPlaceholder" BrowseNames: <TriggerInputs>, <TriggerOutputs>, <DataOutputs>, <DataInputs>, <ConfigurationParams>, <TuningParams>, <StatusVariables>, and <OperationalParams> [CRQ-P6.2-0048].

Table 12: FunctionBlockType Definition

Attribute	Value
BrowseName	FunctionBlockType
IsAbstract	FALSE

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

References	NodeClass	BrowseName	DataType	TypeDefinition	Modeling Rule
Sub-type of 0:BaseObjectType defined in OPC UA Part 5.					
0:HasSubtype	ObjectType	OCFB_Type	Defined in Section 6.4.9.		
0:HasComponent	Object	<FunctionBlocks>		FunctionBlockType	OptionalPlaceholder
0:HasComponent	Variable	Error	FBErrorType	0:BaseDataVariable	Optional
0:HasComponent	Variable	ExtendedError	ExtErrorStructure	0:BaseDataVariable	Optional
0:HasComponent	Variable	Mode	0:UInt	FBModeType	Optional
0:HasComponent	Object	Options		FBOptionsType	Optional
0:HasComponent	Variable	<TriggerInputs>	0:UInt32	TriggerType	OptionalPlaceholder
0:HasComponent	Variable	<TriggerOutputs>	0:UInt32	TriggerType	OptionalPlaceholder
0:HasComponent	Variable	<DataOutputs>	0:BaseDataType	0:BaseDataVariableType	OptionalPlaceholder
0:HasComponent	Variable	<DataInputs>	0:BaseDataType	0:BaseDataVariableType	OptionalPlaceholder
0:HasComponent	Variable	<ConfigurationParams>	0:BaseDataType	0:BaseDataVariableType	OptionalPlaceholder
0:HasComponent	Variable	<TuningParams>	0:BaseDataType	0:BaseDataVariableType	OptionalPlaceholder
0:HasComponent	Variable	<StatusVariables>	0:BaseDataType	0:BaseDataVariableType	OptionalPlaceholder
0:HasComponent	Variable	<OperationalParams>	0:BaseDataType	0:BaseDataVariableType	OptionalPlaceholder
0:HasComponent	Object	Inputs		2:FunctionalGroupType	Mandatory
0:HasComponent	Object	Outputs		2:FunctionalGroupType	Mandatory
0:HasComponent	Object	Status		2:FunctionalGroupType	Mandatory

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Attribute	Value				
BrowseName	FunctionBlockType				
IsAbstract	FALSE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
0:HasComponent	Object	Operational		2:FunctionalGroupType	Mandatory
0:HasComponent	Object	Configuration		2:FunctionalGroupType	Mandatory
0:HasComponent	Object	Tuning		2:FunctionalGroupType	Mandatory

ApplicationName can be used to identify the application to which the instance of the function block belongs.

Error is an instance of *FBErrorType*, sub-type of OPC UA *OptionSetType*, defined by a standard set of errors, but users can add their own.

ExtendedError allows sub-types to provide more detailed information of the errors.

Table 13: O-PAS Functional Group Descriptions

Object	Description
Configuration	Parameters representing the configuration items of the *TopologyElement*, defined in OPC UA Part 5. If the *CurrentWrite* bit is set in the *AccessLevel* attribute, they can be modified by clients that have the correct permissions.
Tuning	Parameters and methods to optimize the behavior of the *TopologyElement*.
Status	Parameters which describe the general health of the *TopologyElement*. This can include diagnostic parameters.
Operational	Parameters and methods useful during normal operation, such as process data.

Function blocks with non-hierarchical references are shown in Table 14.

Table 14: Function Blocks Functional Group Non-Hierarchal References

Source Path	ReferenceType	Is Forward	Target Path
Inputs	Organizes	True	<DataInputs> or <TriggerInputs>
Outputs	Organizes	True	<DataOutputs> or <TriggerOutputs>
Tuning	Organizes	True	<TuningParams>

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Source Path	ReferenceType	Is Forward	Target Path
Configuration	Organizes	True	<ConfigurationParams>
Status	Organizes	True	<StatusVariables>
Operational	Organizes	True	<OperationalVariables> or <MethodCall>

The components of the <DataInputs> and <TriggerInputs> have additional non-hierarchical references which are defined in Table 15.

Table 15: Function Blocks DataInputs and TriggerInputs Additional References

Source Path	ReferenceType	Is Forward	Target Path
<DataInputs>	ConnectedToOutput	True	<DataOutputs>
<TriggerInputs>	ConnectedToOutput	True	<TriggerOutputs>

The additional non-hierarchical references above provide linkages from block inputs to block outputs and trigger inputs to trigger outputs.

The *ExtErrorStructure* structure contains an error code and the string description of the error. The structure is defined in Table 16.

Table 16: ExtErrorStructure Structure

Name	Type	Description
ExtErrorStructure	structure	Sub-type of structure defined in OPC UA Part 5.
ErrorCode	0:Uint16	The function block extended error code.
ErrorString	0:LocalizedText	The function block extended string.

The *ExtErrorStructure* representation in the *AddressSpace* is defined in Table 17.

Table 17: ExtErrorStructure Definition

Attribute	Value				
BrowseName	ExtErrorStructure				
IsAbstract	FALSE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of the structure defined in OPC UA Part 5.					

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

6.4.9 FBErrorType

The DataType defined flags for *FBErrorType* are formally defined in Table 18.

Table 18: FBErrorType Values

Value	Bit No.	Description
OutOfService	0	The block has been placed in the out-of-service mode.
ConfigurationError	1	There is an error with the block's configuration.
SimulationActive	2	The *Process* variable is in simulation mode.
PvHasBadStatus	3	The *Process* variable has bad status.

The *FBErrorType* representation in the *AddressSpace* is defined in Table 19.

Table 19:FBErrorType Definition

Attribute	Value				
BrowseName	FBErrorType				
IsAbstract	FALSE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of 0:OptionSetType defined in OPC UA Part 5.					
0:HasProperty	Variable	0:OptionSetValues	0:LocalizedText [ ]	0:PropertyType	Mandatory

6.4.10 FBOptionsType

The *FBOptionsType* is an abstract type that provides common functionality options used by various function block types. Each function block type will define its specific option type by deriving from this base type and change the modeling rule for the appropriate option(s) to Mandatory. The *FBOptionsType* is formally defined in Table 20.

Table 20: FBOptionsType Definition

Attribute	Value				
BrowseName	FBOptionsType				
IsAbstract	TRUE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of 0:BaseObjectType from OPC UA Part 5.					

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Attribute	Value				
BrowseName	FBOptionsType				
IsAbstract	TRUE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
0:HasProperty	Variable	ActOnIR	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	BadIfLimited	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	BypassEnable	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	DirectActing	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	FaultStateEnable	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	FaultStateToValue	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	IFSIfBadCasIn	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	IFSIfBadIn	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	IFSIfBadTrkInD	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	IncreaseToClose	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	Invert	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	LowCutEnable	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	NoOutLimitsInManual	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	ObeySpLimitsIfCasOrRCas	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	PropagateFaultBackward	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	PropagateFaultForward	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	ReadBackIn	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	SISLatchFaultState	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	SpPvTrackInLO	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	SpPvTrackInLOorIMan	0:Boolean	0:PropertyType	Optional

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Attribute	Value				
BrowseName	FBOptionsType				
IsAbstract	TRUE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
0:HasProperty	Variable	SpPvTrackInManual	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	SpPvTrackReturn	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	SpTrackRetainedTarget	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	TargetManIfFaultStateActive	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	TargetToManualIfBadIn	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	TargetToManualIfBadTrkInD	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	TargetToManualIfFaultStateActive	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	TargetToNextPermittedModeIfBadCasIn	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	TrackEnable	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	TrackIfBadTrkInD	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	TrackInManual	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	UncertainIfLimited	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	UncertainIfManual	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	UnitsConversion	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	UseBackCalOutWithIn	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	UseFaultStateValueOnRestart	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	UsePvForBackCalOut	0:Boolean	0:PropertyType	Optional
0:HasProperty	Variable	UseUncertainAsGood	0:Boolean	0:PropertyType	Optional

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Attribute	Value				
BrowseName	FBOptionsType				
IsAbstract	TRUE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
0:HasProperty	Variable	WriteLock	0:Boolean	0:PropertyType	Optional

Explanations of the *FunctionBlockOptions* are provided in the function block-specific sections where their use is Mandatory.

6.4.11 OCFB_Type

The abstract *OCFB_Type* is a realization of the SIFB defined in IEC 61499 as communication function blocks. The *TagVariableAlias* property is used by an execution engine to register or discover a signal on the OCF.

Table 21: OCFB_Type Definition

Attribute	Value				
BrowseName	OCFB_Type				
IsAbstract	TRUE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of FunctionBlockType.					
0:HasSubtype		OCFB_OutType			
0:HasSubtype		OCFB_InType			
0:HasProperty	Variable	TagVariableAlias	0:String	0:PropertyType	Optional

6.4.12 OCFB_OutType

The *OCFB_OutType* is a sub-type of *FunctionBlockType* defined in Section 6.4.8. It is used to make an O-PAS Layer-F application's *DataOutput* available as an OCF signal.

Table 22: OCFB_OutType Definition

Attribute	Value
BrowseName	OCFB_OutType
IsAbstract	FALSE

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

References	NodeClass	BrowseName	DataType	TypeDefinition	Modeling Rule
Sub-type of OCFB_Type.					
HasComponent	Variable	OutgoingVariable	BaseData Type	0:BaseDataVariableType	Mandatory

6.4.13 OCFB_InType

The *OCFB_InType* is a sub-type of *FunctionBlockType* defined in Section 6.4.8. It is used to connect to an OCF signal, making the signal's data available to an O-PAS Layer-F application.

Table 23: OCFB_InType Definition

Attribute	Value				
BrowseName	OCFB_InType				
IsAbstract	FALSE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of OCFB_Type.					
HasComponent	Variable	ReceivedValue	BaseData Type	0:BaseDataVariableType	Mandatory

6.5 O-PAS OPC UA VariableTypes

The following sections define the O-PAS OPC UA *VariableTypes*.[2]

6.5.1 TriggerType

The *TriggerType* is a concrete *VariableType* that can be used directly. It is a sub-type of *BaseDataVariableType* defined in OPC UA Part 5.

Table 24: TriggerType Definition

Attribute	Value
BrowseName	TriggerType
IsAbstract	FALSE
ValueRank	-1 (-1=Scalar)
DataType	UInt32

[2] For definitions of OPC UA base objects, refer to the corresponding standards at https://opcfoundation.org.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

References	NodeClass	BrowseName	DataType	TypeDefinition	Modeling Rule
Sub-type of 0:BaseDataVariableType defined in OPC UA Part 5.					

The *Value* shall be incremented by 1 for each activation of the trigger. When it reaches the maximum value of 0xFFFFFFFF it shall overflow to 0. The transition from the maximum back to 0 is an occurrence of the trigger. Subscribers should consider each change of the value as the trigger occurring, no matter the quantity of change. For example, a change from 3 to 12 is one trigger, as is a change from 4 to 5 [CRQ-P6.2-0014].

The *SourceTimestamp* of the value shall be updated each time a trigger occurs [CRQ-P6.2-0015].

6.5.2 OPASAnalogSignalType

The *OPASAnalogSignalType* is a sub-type of *AnalogSignalVariableType* defined in the PA-DIM standard. It is formally defined in Table 25. The *ActualValue* variable is extended from a *BaseDataVariableType* to an *AnalogUnitRangeType*. The introduction of the *AnalogUnitRangeType* adds an engineering unit and range to *ActualValue*. When the signal is used as a block input, these additional properties are used to represent the engineering units and range of the signal which is connected to the input. The additional engineering unit is intended to be used for validation and diagnostic purposes to ensure the input's signal source is of the correct engineering units.

The *EngineeringUnits* and *EURange* used by a block for computation and logic are the properties of *OPASAnalogSignalType* itself which it inherits from *0:AnalogUnitRangeType* and they are commonly set to a fixed value when configuring the block. Similarly, the value, timestamp, and status used by the block for computations and logic are also those of the value, source timestamp, and status attributes of the *OPASAnalogSignalType* itself which is provided by either the *ActualValue* or the *SimulationValue* based on the value of the *SimulationState* property.

Note that instances of *OPASAnalogSignalType* are not expected to have the optional variables for simulation present. If simulation is desired, the sub-type *OPASAnalogSimulationSignalType* is used.

Comparing the range and units of an *OPASAnalogSignalType* instance with the range and units of the *ActualValue*, there is a mismatch between what is expected based on the configuration *versus* what is actually arriving at the block. This can help detect misconfigurations, and blocks can provide errors alerting the user if there is a mismatch.

Because it inherits from *0:AnalogUnitRangeType*, *OPASAnalogSignalType* is considered an O-PAS signal, as defined in Section 3.2.

Table 25: OPASAnalogSignalType Definition

Attribute	Value
BrowseName	OPASAnalogSignalType
IsAbstract	FALSE

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

ValueRank	-2 (-2=’Any’)				
DataType	0:Number				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of 3:AnalogSignalVariableType from the OPC PA-DIM standard.					
0:HasComponent	Variable	3:ActualValue	0:Number	0:AnalogUnitRangeType	Mandatory

6.5.3 OPASAnalogSimulationSignalType

The *OPASAnalogSimulationSignalType* is a sub-type of the *OPASAnalogSignalType*. It is formally defined in Table 26. It makes supporting the PA-DIM simulation logic mandatory by making the properties *SimulationState* and *SimulationValue* mandatory.

The simulation logic used by this variable type is defined by the PA-DIM standard.

Because it inherits from *0:AnalogUnitRangeType*, *OPASAnalogSimulationSignalType* is considered an O-PAS signal as defined in Section 3.2.

Table 26: OPASAnalogSimulationSignalType Definition

Attribute	**Value**				
BrowseName	OPASAnalogSimulationSignalType				
IsAbstract	FALSE				
ValueRank	-2 (-2=’Any’)				
DataType	0:Number				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of the OPASAnalogSignalType.					
0:HasComponent	Variable	3:SimulationValue	0:Number	0:BaseDataVariableType	Mandatory
0:HasComponent	Variable	3:SimulationState	0:Boolean	0:BaseDataVariableType	Mandatory

6.5.4 AnalogUnitRangeScaleType

The *AnalogUnitRangeScaleType* is a sub-type of the *0:AnalogUnitRangeType*. It is formally defined in Table 27. It adds a second set of range and engineering units that can be used for scaling, while the native *EURange* and *EngineeringUnits* properties inherited from *AnalogUnitRangeType* can be used to describe the actual output value which might differ in range and units; e.g., due to a certain block mode.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Because it inherits from *0:AnalogUnitRangeType*, *AnalogUnitRangeScaleType* is considered an O-PAS signal as defined in Section 3.2.

Table 27: AnalogUnitRangeScaleType Definition

Attribute	**Value**				
BrowseName	AnalogUnitRangeScaleType				
IsAbstract	FALSE				
ValueRank	-2 (-2=’Any’)				
DataType	0:Number				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of the 0:AnalogUnitRangeType from OPC UA Part 8.					
0:HasProperty	Variable	ScalingRange	0:Range	0:PropertyType	Mandatory
0:HasProperty	Variable	ScalingEngineering Units	0:EUInformation	0:PropertyType	Mandatory

ScalingRange and *ScalingEngineeringUnits* are configurable variables used to scale the value of the *AnalogUnitRangeScaleType* variable. Sometimes blocks have modes in which this scaling is bypassed (e.g., a direct mode). In this case, the *EngineeringUnits* and *EURange* don’t match *ScalingEngineeringUnits* and *ScalingRange*.

6.5.5 FBModeType

The *FBModeType* represents the current operating mode of a function block. The *FBModeType* is a sub-type of *MultiStateValueDiscreteType* defined in OPC UA Part 8, inheriting the *EnumValues* and *ValueAsText* properties from the base type. *EnumValues* is an array of *EnumValueType*. Each entry of the array represents one enumeration value with its integer notation, display name, and description. This represents enumerations with integers that are not zero-based or have gaps (e.g., 1, 2, 4, 8, 16). Variable instances of *FBModeType* expose the current unsigned integer notation in their *Value* attribute. The reserved values for the *EnumValues* property are defined in Table 29. All *EnumValues* less than 100 are reserved for O-PAS future use. The *FBModeType* is formally defined in Table 28.

Table 28: FBModeType Definition

Attribute	**Value**
BrowseName	FBModeType
IsAbstract	FALSE
ValueRank	-1 (-1=Scalar)
DataType	UInt

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

References	NodeClass	BrowseName	DataType	TypeDefinition	Modeling Rule
Sub-type of 0:MultiStateValueDiscreteType defined in OPC UA Part 8.					
0:HasProperty	Variable	Normal	UInt	0: PropertyType	Optional
0:HasComponent	Variable	Target	UInt	0:MultiStateValueDiscreteType	Mandatory

The *Value* of this VariableType provides the function block actual mode. The optional *Normal* property provides the cold start default mode. The *Target* is the function block requested mode.

The *EnumValues* for "Target" mode shall be the set of permitted modes that is a subset of available modes, as defined by the *EnumValues* of *FBModeType* [CRQ-P6.2-0042].

Reserved values for *FBModeType EnumValues* are defined in Table 29.

Table 29: FBModeType Reserved EnumValues

Name	Value	Description
OutOfService	0	The block is disabled and not being computed.
InitializationManual	1	Output or control blocks use a back calculation input variable to initialize their calculated output variable.
LocalOverride	2	Output or control blocks use an input variable for the purpose of output tracking.
Manual	3	The block's outputs are manually set.
Automatic	4	The block is executing with local control; e.g., setpoint from operator via HMI.
Cascade	5	The block is executing with control by another function block; e.g., setpoint from an upstream master control block in application.
RemoteCascade	6	The block is executing with control from a remote resource; e.g., setpoint from an application on a different node.

6.6 O-PAS OPC UA ReferenceTypes

6.6.1 ConnectedToOutput

The *ConnectedToOutput* ReferenceType is a concrete ReferenceType that can be used directly. In this document, *ConnectedToOutput* is used to model the connection between inputs and outputs. It is a sub-type of the *NonHierarchicalReferences* ReferenceType defined in OPC UA Part 5.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Table 30: ConnectedToOutput ReferenceType Definition

Attribute	Value				
BrowseName	ConnectedToOutput				
InverseName	ConnectedFromInput				
Symmetric	FALSE				
IsAbstract	FALSE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of the 0:NonHierarchicalReferences ReferenceType defined in OPC UA Part 5.					

6.7 Instance AddressSpace

6.7.1 OPAS Folder and SignalType Alias

As specified in O-PAS Part 4 – O-PAS Connectivity Framework (OCF), the "OPAS" folder shall be used as the root folder for all O-PAS related objects. The entry point for an O-PAS UA client is the O-PAS folder. Therefore, all *DCNBaseType* objects shall be organized by this root O-PAS folder [CRQ-P6.2-0016].

If configured, O-PAS *SignalType* instances shall register the *Tag* variable to the *Objects.Aliases.Tags* (refer to OPC UA Part 17 for more information on alias) [CRQ-P6.2-0017].

6.7.2 FunctionBlocks Aliases

In OPC UA Part 17, three standard *AliasNameCategoryType* objects are defined for maintaining the hierarchical structure of *AliasNameCategories* in an OPC UA server; see Figure 13. They are:

- *Aliases* – this is the root folder for all alias-related objects on an OPC UA server

  It can contain instances of *AliasNameType* objects and/or instances of *AliasNameCategoryType* objects.

- *TagVariable*s – this is a folder containing only *AliasNameType* instances that contains an *AliasFor* reference to variables, where the *AliasNameType* instance *AliasFor* target can be directly subscribed to by a client

  It can contain additional *AliasNameCategoryType* objects, which could be used to create a hierarchy or other structure. A single instance of *AliasNameType* can exist in more than one location in the tree.

- *Topics* – this instance of *AliasNameCategoryType* is optional, but if the *Topics* instance is present it is the root folder for all instances of *AliasNameType* that contain an *AliasFor* reference to a publish/subscribe configuration

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

This instance restricts instances of *AliasNameType* to have only *AliasFor* references that point to a published dataset object. It can contain additional *AliasNameCategoryType* objects, which could be used to create a hierarchy or other structure. A single instance of *AliasNameType* can exist in more than one location in the tree.

Defined in this document, *FunctionBlocks* is an instance of *AliasNameCategoryType*. It shall only contain instances of *AliasNameType* that have *AliasFor* references to instances of O-PAS *FuntionBlockType* or its sub-type [CRQ-P6.2-0018]. It can also contain additional *AliasNameCategoryType* objects, which could be used to create a hierarchy or other structure. The *FunctionBlocks* instance is formally defined in Table 31.

Table 31: FunctionBlocks AliasNameCategoryType Definition

Attribute	**Value**				
BrowseName	FunctionBlocks				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Organized by the Aliases folder defined in OPC UA Part 5.					
0:HasTypeDefinition	ObjectType	AliasNameCategoryType	Defined in OPC UA Part 17.		

Figure 13 shows an example of how *Aliases* for process variables and function blocks, from multiple OPC UA servers, are aggregated and organized in an aggregating server or a GDS. Refer to OPC UA Part 17 for more details.

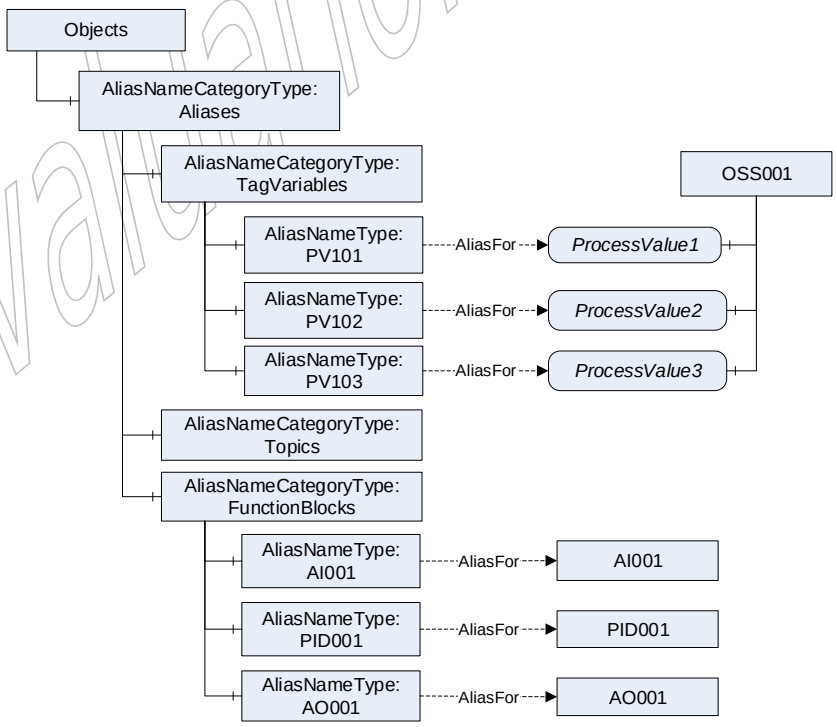


Figure 13: Example of O-PAS Function Blocks AliasNameCategoryType

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

6.8 Profiles and Facets

This section defines the Profiles and Facets for the O-PAS Basic Information Model.

Table 32: Profiles and Facets URIs

Profile/Facet	Related Category	URI
OBC-202 – O-PAS I/O Server Profile	Server	https://www.opengroup.org/open-process-automation/O-PAS/IOServer
OBC-203 – O-PAS Function Block Compute Server Profile	Server	https://www.opengroup.org/open-process-automation/O-PAS/FBComputeServer
OBC-F-201 – O-PAS Base Function Block Facet	Server	https://www.opengroup.org/open-process-automation/O-PAS/BaseFB

6.8.1 OBC-F-201: O-PAS Base Function Block Facet

The OBC-F-201 Facet defines the O-PAS Base Function Block.

Implementing the OBC-F-201 Facet requires the support of the ObjectTypes and VariableTypes listed in Table 33. O-PAS products that implement the OBC-F-201 Facet shall meet the ObjectType and VariableType definitions described in Table 33. O-PAS products that implement the OBC-F-201 Facet shall also meet all Conformance Requirements identified in the section(s) where the ObjectTypes and VariableTypes are defined; i.e., the Reference column in Table 33.

Table 33: OBC-F-201: O-PAS Base Function Block Facet ObjectType Requirements

CRQ Identifier	ObjectType/Variable	Type	Reference	Mandatory/ Optional
CRQ-P6.2-0033	Implement/extend FunctionBlockType	ObjectType	Section 6.4.8	M
CRQ-P6.2-0034	Implement/extend FBOptionsType	ObjectType	Section 6.4.8	M
CRQ-P6.2-0035	Implement/extend TriggerType	VariableType	Section 6.4.8	O
CRQ-P6.2-0036	Implement/extend FBModeType	VariableType	Section 6.5.3	M

6.8.2 OBC-202: O-PAS I/O Server Profile

The OBC-202 Profile extends the base functionality of the OCF-001 (and optionally OCF-002) Profile defined in O-PAS Part 4 to specify the minimum OPC UA server facets available for servers that implement O-PAS Part 6.2. O-PAS products that implement OBC-202 shall meet the requirements described in Table 34.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Table 34: OBC-202: O-PAS I/O Server Requirements

CRQ Identifier	Profile/Facet/Requirement	Defined in	Mandatory/ Optional
CRQ-P6.2-0039	SEC-F-001: All ANSI/ISA-62443-4-2 requirements	O-PAS Part 2	M
CRQ-P6.2-0019	OCF-001: OPC UA Client/Server Profile	O-PAS Part 4	M
CRQ-P6.2-0020	OCF-002: OPC UA Client/Server and PubSub Profile	O-PAS Part 4	O
CRQ-P6.2-0016	CRQ-P6.2-0016: DCNBaseType Instance AddressSpace	Section 6.4.1	M
CRQ-P6.2-0017	CRQ-P6.2-0017: O-PAS Signal Tags Alias	Section 6.4.3	M
See Table 35 for additional requirements.			

OBC-202 products shall support all requirements listed in Section 6.7 [CRQ-P6.2-0046].

O-PAS products that implement the OBC-202 Profile require several objects to represent DCN and I/O server functionality: they shall meet the ObjectType definitions described in Table 35Table 33. They shall also meet all Conformance Requirements identified in the section(s) of this document where the ObjectTypes are defined; i.e., the sections listed in the ObjectType Reference column in Table 35.

Table 35: OBC-202: O-PAS I/O Server ObjectType Requirements

CRQ Identifier	ObjectType	ObjectType Reference	Mandatory/ Optional
CRQ-P6.2-0021	Implement/extend DCNBaseType	Section 6.4.1	M
CRQ-P6.2-0022	Implement/extend OPASBaseEngineType	Section 6.4.3	M
CRQ-P6.2-0023	Implement/extend IOServiceEngineType	Section 6.4.5	M
CRQ-P6.2-0024	Implement/extend ISignalServiceType	Section 6.4.4	M

6.8.3 OBC-203: O-PAS Function Block Compute Server Profile

Table 36 and Table 37 specify the minimum server profile available for servers that implement O-PAS Part 6.2. O-PAS products that implement OBC-203 shall meet the requirements described in Table 36 and Table 37.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Table 36: OBC-203: O-PAS Function Block Compute Server Requirements

CRQ Identifier	Profile/Facet/Requirement	Defined in	Mandatory/ Optional
CRQ-P6.2-0043	OBC-F-201: O-PAS Base Function Block Facet	Section 6.8.2	M
CRQ-P6.2-0040	SEC-F-001: All ANSI/ISA-62443-4-2 requirements	O-PAS Part 2	M
CRQ-P6.2-0044	OCF-001: OPC UA Client/Server Profile	O-PAS Part 4	M
CRQ-P6.2-0045	OCF-002: OPC UA Client/Server and PubSub Profile	O-PAS Part 4	O
CRQ-P6.2-0016	CRQ-P6.2-0016: DCNBaseType Instance AddressSpace	Section 6.4.1	M
CRQ-P6.2-0017	CRQ-P6.2-0017: O-PAS Signal Tags Alias	Section 6.4.4	M
CRQ-P6.2-0018	CRQ-P6.2-0018: O-PAS FunctionBlocks Alias	Section 6.7.2	M
See Table 37 for additional requirements.			

OBC-203 products shall support all requirements listed in Section 6.7 [CRQ-P6.2-0047].

O-PAS products that implement the OBC-203 Profile require several objects to represent the DCN and Function Block Compute Server functionality.

Implementing the OBC-203 Profile requires the support of the ObjectTypes listed in Table 37. O-PAS products that implement the OBC-203 Profile shall provide the ObjectType definitions described in Table 37Table 33. O-PAS products that implement the OBC-203 Profile shall also meet all Conformance Requirements identified in the section(s) where these ObjectTypes are defined; i.e., the sections listed in the ObjectType Reference column of Table 37.

Table 37: OBC-203: O-PAS Function Block Compute Server ObjectType Requirements

CRQ Identifier	ObjectType	ObjectType Reference	Mandatory/ Optional
CRQ-P6.2-0025	Implement/extend DCNBaseType	Section 6.4.1	M
CRQ-P6.2-0026	Implement/extend OPASBaseEngineType	Section 6.4.3	M
CRQ-P6.2-0027	Implement/extend FBExecutionEngineType	Section 6.4.6	M
CRQ-P6.2-0028	Implement/extend ISignalServiceType	Section 6.4.4	M
CRQ-P6.2-0029	Implement/extend FBGroupType	Section 6.4.6	M
CRQ-P6.2-0030	Implement/extend OCFB_Type	Section 6.4.11	M
CRQ-P6.2-0031	Implement/extend OCFB_OutType	Section 6.4.12	M

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

CRQ Identifier	ObjectType	ObjectType Reference	Mandatory/ Optional
CRQ-P6.2-0032	Implement/extend OCFB_InType	Section 6.4.13	M

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

7 Namespaces

7.1 Namespace Metadata

Table 38 defines the namespace metadata for this document. The *Object* is used to provide version information for the namespace and an indication about static nodes. Static nodes are identical for all attributes in all servers, including the *Value* attribute. See OPC UA Part 5 for more details.

The information is provided as *Object* of type *NamespaceMetadataType*. This *Object* is a component of the *Namespaces* object that is part of the *Server* object. The *NamespaceMetadataType* ObjectType and its properties are defined in OPC UA Part 5.

The version information is also provided as part of the *ModelTableEntry* in the *UANodeSet* XML file. The *UANodeSet* XML schema is defined in OPC UA Part 6.

Table 38: NamespaceMetadata Object for this Document

Attribute	**Value**		
BrowseName	https://www.opengroup.org/open-process-automation/O-PAS/Base		
References	**BrowseName**	**DataType**	**Value**
HasProperty	NamespaceUri	String	https://www.opengroup.org/open-process-automation/O-PAS/Base
HasProperty	NamespaceVersion	String	2.1
HasProperty	NamespacePublicationDate	DateTime	2021-05-17
HasProperty	IsNamespaceSubset	Boolean	True
HasProperty	StaticNodeIdTypes	IdType []	Integer
HasProperty	StaticNumericNodeIdRange	NumericRange []	0,7000
HasProperty	StaticStringNodeIdPattern	String	Null

7.2 Handling of OPC UA Namespaces

Namespaces are used by the OPC UA to create unique identifiers across different naming authorities. The attributes NodeId and BrowseName are identifiers. A node in the UA *AddressSpace* is unambiguously identified using a NodeId. Unlike NodeIds, the BrowseName cannot be used to unambiguously identify a node. Different nodes may have the same

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

BrowseName. They are used to build a BrowsePath between two nodes or to define a standard *Property*.

Servers may often choose to use the same namespace for the NodeId and the BrowseName. However, if they want to provide a standard property, the BrowseName must have the namespace of the standards body, although the namespace of the NodeId reflects something else; for example, the *EngineeringUnits* property. All NodeIds of nodes not defined in this document shall not use the standard namespaces [CRQ-P6.2-0038].

Table 39 provides a list of mandatory and optional namespaces used in an O-PAS OPC UA server.

Table 39: Namespaces Used in an O-PAS Server

NamespaceURI	Description	Use
http://opcfoundation.org/UA/	Namespace for NodeIds and BrowseNames defined in the OPC UA specification. This namespace must have namespace index 0.	Mandatory
Local Server URI	Namespace for nodes defined in the local server. This namespace must have namespace index 1.	Mandatory
http://opcfoundation.org/UA/DI/	Namespace for NodeIds and BrowseNames defined in OPC UA Part 100. The namespace index is server-specific.	Mandatory
http://opcfoundation.org/UA/Dictionary/IRDI	Namespace for NodeIds of International Registration Data Identifiers (IRDIs) using *HasDictionaryEntry*. The namespace index is server-specific.	Optional
http://opcfoundation.org/UA/PADIM/	Namespace for NodeIds and BrowseNames defined in the OPC PA-DIM standard. The namespace index is server-specific.	Mandatory
https://www.opengroup.org/open-process-automation/O-PAS/Base	Namespace for NodeIds and BrowseNames defined in this document. The namespace index is server-specific.	Mandatory
Vendor-specific types	A server may provide vendor-specific types like types derived from ObjectTypes defined in this document in a vendor-specific namespace.	Optional
Vendor-specific instances	A server provides vendor-specific instances of the standard types or vendor-specific instances of vendor-specific types in a vendor-specific namespace. It is recommended to separate vendor-specific types and vendor-specific instances into two or more namespaces.	Mandatory

Table 40 provides a list of namespaces and their index used for BrowseNames in this document. The default namespace of this document is not listed since all BrowseNames without a prefix use this default namespace.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Table 40: Namespaces Used in this Document

NamespaceURI	Namespace Index	Example
http://opcfoundation.org/UA/	0	0:EngineeringUnits
http://opcfoundation.org/UA/DI/	2	2:DeviceRevision
http://opcfoundation.org/UA/PADIM/	3	3:SimulationValue

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

A Connectivity Framework Namespace and Mappings (Normative)

A.1 Namespace and Identifiers for the Connectivity Framework Information Model

This appendix defines the numeric identifiers for all of the numeric *NodeIds* defined in the O-PAS Standard. The identifiers are specified in a CSV file with the following syntax:

```
<SymbolName>, <Identifier>, <NodeClass>
```

where the *SymbolName* is either the *BrowseName* of a *Type* node or the *BrowsePath* for an *Instance* node that appears in the specification and the *Identifier* is the numeric value for the *NodeId*. Multiple *NodeIds* in the same CSV file are separated by newline characters.

The *BrowsePath* for an *Instance* node is constructed by appending the *BrowseName* of the *Instance* node to the *BrowseName* for the containing instance or type. An underscore character is used to separate each *BrowseName* in the path.

The *NamespaceUri* for all *NodeIds* defined here is: https://www.opengroup.org/open-process-automation/O-PAS/Base.

The CSV of O-PAS numeric *NodeId* values released with the O-PAS Standard, Version 2.1 can be found here:

http://open-process-automation.projects.opengroup.org/twg/part-6-information-models/part-6.2-basic-configuration/DOCX/v02.01.00prelim/opas_part_6_2_nodesetIds.csv

A computer-processable version of the complete information model defined in the O-PAS Standard is also provided. It follows the XML information model schema syntax defined in the OPC UA Part 6.

The information model schema released with the O-PAS Standard, Version 2.1 can be found here:

http://open-process-automation.projects.opengroup.org/twg/part-6-information-models/part-6.2-basic-configuration/DOCX/v02.01.00prelim/opas_part_6_2_nodeset2.xml

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

B O-PAS AutomationML Definitions (Normative)

The normative definitions of O-PAS AutomationML (AML) are available in the AML files here:

http://open-process-automation.projects.opengroup.org/twg/part-6-information-models/part-6.2-basic-configuration/DOCX/v02.01.00prelim/opas_part_6_2.aml

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

C Future Extension to IOServiceEngineType (Informative)

Figure 14 illustrates planned extension to the *IOServiceEngineType* in a future version of the O-PAS Standard to include the ITransducerServiceType. The transducer service is envisioned to include the following functionalities:

- Loading of field device profiles/configurations
- Execute methods
- Calibration
- Field Device Integration (FDI) port/interactions

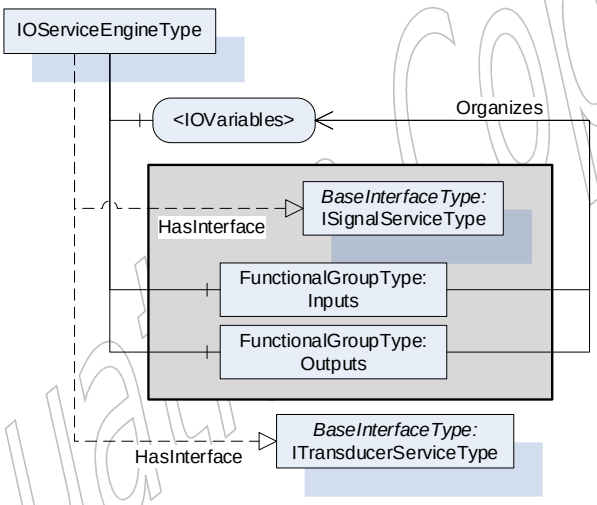


Figure 14: OCF I/O Service with Transducer Service Interface

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

D Abbreviations

For the OPAF Glossary and Abbreviations, see the definitions at: www.opengroup.org/open-process-automation-forum/glossary.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Index

.opas file 15
.opas file format 15
alias 9
AML 6, 55
AnalogUnitRangeScaleType 42
AutomationML 55
Basic-BaseEngine 18
Basic-ControlApplication 18
Basic-DCN 17
Basic-DCP 17
Basic-FBExecutionEngine 18
Basic-FunctionBlock 19
Basic-FunctionBlockApplication 18
Basic-FunctionBlockGroup 19
Basic-IOServiceEngine 19
Basic-OCFB_IN 20
Basic-OCFB_OUT 20
Basic-Project 16
Basic-Signal 16
Basic-SignalInterface 16
Basic-SignalService 19
Basic-TagList 17
Basic-TagNameplate 17
Basic-TriggerInterface 17
Basic-VendorNameplate 17
common models 6
common objects 6
configuration constraints 14
conformance 1
ConnectedToOutput 44
control application 10
data input 13
data output 13
DCN 1, 7
DCNBaseType 28
DCP 7
download/upload 22
download/upload model 1
FBErrorType 36
FBExecutionEngineType 31
FBGroupType 31
FBModeType 43
FBOptionsType 36
FieldComm Group 8
function block 11, 12, 13
FunctionBlockType 32
GDS 3
HMI 8
IDCNBaseType 27
import/export 22
import/export format 1, 15
information model 1, 6
IOServiceEngineType 30
ISignalServiceType 30
Layer-F 1
namespaces 54
OBC-201 21
OBC-202 47
OBC-203 48
OBC-F-101 21
OBC-F-201 47
ObjectType 27
OCF 1
OCFB 10
OCFB_InType 40
OCFB_OutType 39
OCFB_Type 39
O-PAS common objects 15
O-PAS engine 7
O-PAS folder 45
O-PAS Information Model 23
O-PAS signals 8
OPASAnalogSignalType 41
OPASAnalogSimulationSignalType 42
OPASBaseEngineType 29
OPC Foundation 8
OPC UA Information Model 26
Open Packaging Conventions 15
PA-DIM 8, 25
PID loop 11
POU 32
runtime model 7
SIFB 10, 39
transducer service 56
trigger input 13
trigger output 13
TriggerType 40
UA-Tag 16
UA-TagVariable 16
UA-UnitOfMeasure 21
UA-UOMLibrary 20
UA-UOMLInk 21
UA-UOMNamespace 21
UML 1
UML model 6

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution